

UNIVERSIDAD DE SEVILLA
INSTITUTO DE MICROELECTRÓNICA DE SEVILLA (IMSE-CNM-CSIC)

MÁSTER UNIVERSITARIO EN MICROELECTRÓNICA:
DISEÑO Y APLICACIONES DE SISTEMAS MICRO/NANOMÉTRICOS

**Design of a Key-Generation Cryptographic
Engine for Integrated-Circuit Implementation:
a Transistor-Level Characterized Entropy
Source with FPGA Verification against an
ESP32**

Master's Thesis

Author:
Mario García Jiménez

Supervisor:
[To be assigned]

Academic year 2025–2026

Abstract

Cryptographic keys are only as good as the entropy behind them, yet most embedded key generators are evaluated as black boxes: statistical test suites are run on their output and a pass is taken as evidence of randomness. The author's previous work on the hardware random number generator of the ESP32 showed why that is not enough — a source with low min-entropy can pass NIST SP 800-22 — and NIST itself has since rejected the suite for assessing cryptographic generators. This thesis takes the opposite route: a white-box key-generation engine whose entropy is justified from the physics of the device. A ring-oscillator entropy source is designed at transistor level and its jitter is extracted from simulation and turned, through a validated stochastic model, into a lower bound on min-entropy per bit and into the design parameters of the sampler. Around it, a fully portable digital core — SP 800-90B health tests, an AES-based conditioning and CTR_DRBG with derivation function, AES-CMAC authentication and an I2C register interface — is written once and verified three ways: against the official test vectors, in simulation on a Basys 3 FPGA acting as the verification vehicle, and against an ESP32 that recomputes every result with an independent implementation (mbedtls). The same VHDL is finally synthesized on an open 130 nm standard-cell library to give the area in gate equivalents and the path to a dedicated integrated circuit. All code, models, vectors and data are released.

Keywords true random number generator, ring oscillator, jitter, min-entropy, stochastic model, AIS 20/31, SP 800-90B, CTR_DRBG, AES-CMAC, secure element, FPGA, ASIC, ESP32.

Resumen

Las claves criptográficas valen lo que vale la entropía que hay detrás, y sin embargo la mayoría de los generadores embebidos se evalúan como cajas negras: se pasan baterías de tests estadísticos sobre su salida y un aprobado se toma como prueba de aleatoriedad. El trabajo previo del autor sobre el generador hardware del ESP32 mostró por qué no basta — una fuente con poca min-entropía puede pasar NIST SP 800-22 — y el propio NIST ha rechazado después esa batería para evaluar generadores criptográficos. Este TFM toma el camino contrario: un motor de generación de claves de caja blanca cuya entropía se justifica desde la física del dispositivo. Se diseña a nivel de transistor una fuente de entropía por oscilador de anillo, se extrae su jitter por simulación y se convierte, mediante un modelo estocástico validado, en una cota inferior de min-entropía por bit y en los parámetros de diseño del muestreador. A su alrededor, un núcleo digital totalmente portable — tests de salud SP 800-90B, acondicionado y CTR_DRBG con función de derivación basados en AES, autenticación AES-CMAC e interfaz de registros I2C — se escribe una sola vez y se verifica de tres maneras: contra los vectores oficiales, en simulación sobre una FPGA Basys 3 que hace de vehículo de verificación, y frente a un ESP32 que recalcula cada resultado con una implementación independiente (mbedtls). El mismo VHDL se sintetiza por último sobre una biblioteca abierta de celdas estándar de 130 nm para dar el área en puertas equivalentes y el camino hacia un circuito integrado dedicado. Todo el código, modelos, vectores y datos se publican.

Acknowledgements

To my family, my colleagues at Airzone, and the faculty of the Máster en Microelectrónica at IMSE-CNM, for their support during this work.

Contents

Abstract	1
Resumen	2
Acknowledgements	3
List of Figures	7
List of Tables	8
List of Acronyms	9
1 Introduction	11
1.1 Motivation: keys are only as good as their entropy	11
1.2 From the black-box ESP32 study to a white-box engine	12
1.3 Objectives	12
1.4 Scope and exclusions	13
1.5 Methodology and reproducibility conventions	14
1.6 Structure of the document	14
2 State of the Art	15
2.1 Ring-oscillator TRNG architectures	15
2.2 Stochastic models and entropy bounds	16
2.3 Embedded jitter measurement	17
2.4 Standards: AIS 20/31 v3.0 and NIST SP 800-90B	18
2.5 Symmetric cores for constrained hardware	19
2.6 Commercial secure elements	19
2.7 From FPGA to ASIC: what changes in the entropy source	20
2.8 Gap addressed by this work	21
3 Entropy Source: from Transistor to Min-Entropy	22
3.1 The elementary ring oscillator	22
3.2 Stochastic model and the quality factor	23

3.3	Exact conditional entropy vs. the asymptotic bound	23
3.3.1	Design target	24
3.4	Embedded jitter estimator and its two corrections	24
3.5	Transistor-level bench and its resolution	25
3.6	Design target and divider selection	26
4	Digital Engine	27
4.1	Architecture and the single-primitive decision	27
4.2	AES-128 encryption core	28
4.3	AES-CMAC	28
4.4	CTR_DRBG with derivation function	29
4.5	Health tests	29
4.6	Capture path and clock-domain crossings	30
4.7	I2C register interface and command sequencer	30
5	Verification	32
5.1	Strategy: vectors, simulation, independent verifier	32
5.2	Reference models in Python	33
5.3	Testbenches and results	33
5.4	ESP32 verifier firmware	35
5.5	FPGA bring-up	35
6	Path to an Integrated Circuit	36
6.1	Standard-cell synthesis flow	36
6.2	Area per block in gate equivalents	36
6.3	Optimizations and their expected savings	37
6.4	What the ring oscillator needs in silicon	38
6.5	Cost and schedule of an academic shuttle	38
7	Results and Comparison	40
7.1	Status against the objectives	40
7.2	Transistor-level noise of the ring	40
7.3	Operating point: divider and throughput	42
7.4	Functional verification, consolidated	43
7.5	Comparison with commercial secure elements	43
7.6	Uncertainty budget	44
7.7	The pending campaign	45
8	Conclusions and Future Work	47
8.1	Conclusions	47
8.2	Contributions	47

8.3 Future work	47
Bibliography	48
A Register Map	62
B Official Test Vectors Used	63
C Reproduction Guide	64

List of Figures

4.1	Data path of the engine. Only the ring oscillators are technology dependent; every other block is portable RTL.	27
-----	---	----

List of Tables

2.1	AIS-compatible TRNG cores on Xilinx Spartan-6 (45 nm), after Petura <i>et al.</i> [8], Table II. Entropy is the T8 estimate; “feasibility” is the authors’ rank (1 best).	16
3.1	Conditional Shannon entropy per bit: exact wrapped-Gaussian computation vs. truncated expansions [SIMULATED].	24
3.2	Recovery of a known Q_{raw} by the corrected embedded estimator ($K = 20\,000$ blocks of $N = 100$ pairs) [SIMULATED].	25
5.1	Testbenches, what they check and their result (GHDL 6.0, VHDL-2008). .	33
6.1	Logic synthesis on IHP SG13G2, flattened, no routing. $1\text{ GE} = 7.2576\text{ }\mu\text{m}^2$. Lower bound on area [SIMULATED].	37
7.1	Objectives and the evidence supporting each.	41
7.2	Period jitter of the five-stage ring against injected noise density [SIMULATED]. $\sigma_{\text{rel}} = \sigma/T$	41
7.3	Divider and resulting throughput per ring for $Q \geq 0.2286$ [ESTIMATED]. . .	42
7.4	Verification totals [SIMULATED].	43
7.5	The engine against commercial secure elements. Certification data from the manufacturers’ datasheets.	44

List of Acronyms

AES	Advanced Encryption Standard
AIS	Anwendungshinweise und Interpretationen zum Schema (BSI)
APT	Adaptive Proportion Test
ASIC	Application-Specific Integrated Circuit
BSI	Bundesamt für Sicherheit in der Informationstechnik
CAVP	Cryptographic Algorithm Validation Program
CDC	Clock Domain Crossing
CMAC	Cipher-based Message Authentication Code
CTR_DRBG	Counter-mode Deterministic Random Bit Generator
DRBG	Deterministic Random Bit Generator
ERO	Elementary Ring Oscillator
FPGA	Field-Programmable Gate Array
GE	Gate Equivalent
I2C	Inter-Integrated Circuit
KAT	Known-Answer Test
LUT	Look-Up Table
MEMS	Micro-Electro-Mechanical System
MPW	Multi-Project Wafer
NIST	National Institute of Standards and Technology
PDK	Process Design Kit

PLL	Phase-Locked Loop
PUF	Physically Unclonable Function
RCT	Repetition Count Test
RO	Ring Oscillator
RTL	Register-Transfer Level
SPI	Serial Peripheral Interface
TRNG	True Random Number Generator
XADC	Xilinx Analog-to-Digital Converter

Chapter 1

Introduction

1.1 Motivation: keys are only as good as their entropy

Every secure connection an embedded device opens, every firmware image it authenticates and every credential it stores rests on a key, and every key rests on a source of randomness. If that source is weaker than assumed, no amount of cryptographic sophistication downstream recovers what was lost at the start: a 128-bit key drawn from a generator with effectively 40 bits of entropy is a 40-bit key. This is not a theoretical concern. Frequency injection on the supply rail of a commercial secure microcontroller collapsed its key space from 2^{64} to about 3300 [1]; a certified smart-card generator was found biased despite passing the standard statistical suites [2]; and the hardware generator of one of the most widely deployed IoT microcontrollers, the ESP32, silently returns pseudo-random values when its radio is off while still passing every black-box test [3].

The common thread is that randomness has been judged by looking at the output. Statistical batteries such as NIST SP 800-22 check whether a bit stream *looks* uniform and independent; they cannot tell a physical noise source from a deterministic generator seeded with a constant, and NIST said so formally in 2022 when it announced the revision of that publication, “rejecting its use for assessing cryptographic random number generators” [4]. The current standards — AIS 20/31 in Europe [5] and SP 800-90B in the United States [6] — ask for something different: a stochastic model of the noise source, an estimate of its min-entropy derived from that model, and health tests that watch the source itself rather than its output. That is a microelectronics problem before it is a cryptography problem, and it is the problem this thesis addresses.

1.2 From the black-box ESP32 study to a white-box engine

This work continues a previous project by the author that characterized the hardware random number generator of the ESP32 as a black box: the same peripheral was exercised under four operating conditions, its output was run through a self-validated implementation of the fifteen tests of SP 800-22, and a key-harvesting scheme combining several on-chip sources was proposed. Its central finding was methodological: the tests passed under conditions in which the entropy could not have been what the tests implied, which is exactly the blind spot described above. Two of its numbers also had to be revisited with hindsight — a temperature correlation was reported from a sensor whose resolution was a single step over the observed range, and a stability figure was inferred where it could have been measured — and both lessons are turned into rules in Section 1.5.

The natural next step was to build the generator instead of measuring someone else’s: an engine whose entropy source is designed, whose jitter is measured inside the device, whose min-entropy follows from a model, and whose keys are produced and used by primitives verified against the standards. The ESP32 changes role from subject to judge. It has its own AES and SHA accelerators and a mature cryptographic library, so it can recompute everything the engine produces with an implementation that shares nothing with it, over a real bus.

The engine is designed for silicon. Secure elements — small integrated circuits that generate, store and use keys on behalf of a host microcontroller over I2C — are a mature product category; the *transparency* of their entropy is not. Of three widely used parts, one ships with a NIST validation of its entropy source and two do not mention SP 800-90 anywhere in their documentation [7]. The FPGA in this work is therefore the verification vehicle, not the target: it lets millions of test vectors run against the design in minutes, which no transistor-level simulator can, before the same description is synthesized on standard cells.

1.3 Objectives

- O1 Design a ring-oscillator entropy source at transistor level, with enable and sampler, and simulate it with noise, stating the numerical resolution of the bench before any figure is read from it.
- O2 Extract the jitter from the simulated waveforms with an estimator that separates quantization, thermal and flicker components, validated beforehand on synthetic data of known jitter.
- O3 Turn the jitter into a min-entropy bound through a stochastic model checked against

an exact computation, and derive the sampler divider that meets the AIS 20/31 target with margin.

- O4 Write the digital engine — health tests, AES-based conditioning and CTR_DRBG, AES-CMAC and an I2C register interface — as technology-independent RTL verified against official vectors.
- O5 Verify the engine functionally on an FPGA with the ESP32 as an independent oracle: known-answer tests, cross interoperability and challenge-response.
- O6 Run an entropy campaign on real silicon (the FPGA’s rings) with the SP 800-90B estimators and compare with the ESP32 generator under the same methodology.
- O7 Synthesize the engine on an open standard-cell library and report its area in gate equivalents with the growth factors to core area.
- O8 Compare the design with the ESP32 generator, with commercial secure elements and with published circuits, with stated uncertainties.

The minimum defensible result is O1–O3 together with O5: an entropy source designed from the transistor, with its jitter extracted and turned into a bound, and a digital engine verified against an independent implementation. A negative result that is well measured closes the work as well as a positive one: if the accumulation exponent on hardware is two rather than one, flicker dominates and the divider cannot grow indefinitely, and that is a finding.

1.4 Scope and exclusions

Four things are deliberately out of scope, each for a stated reason. *Fabrication*: an academic shuttle is affordable and the engine fits in the smallest block offered, but the schedule from tape-out to packaged and characterized samples exceeds a master’s thesis; the cost and the path are documented, not executed. *Asymmetric cryptography*: an elliptic-curve multiplier is a project in itself and would consume the whole area budget; the engine provides symmetric keys and authentication. *Side-channel resistance*: the threat model is a logical, remote adversary; timing leaks are avoided by construction, but no masking or trace-based evaluation is attempted, and the institute’s own laboratory for such work is noted as the natural continuation. *Formal certification*: the criteria of AIS 20/31 and SP 800-90B and the official vectors are used throughout, and no validation is claimed.

1.5 Methodology and reproducibility conventions

Four conventions apply to every figure in this document. First, each number is labelled [MEASURED], [SIMULATED] or [ESTIMATED], and carries its uncertainty or the source it comes from. Second, **the resolution of the instrument is declared before any trend is interpreted**, whether the instrument is a sensor, an estimator or a simulator — the numerical noise floor of the SPICE bench and the quantization of a temperature sensor are treated alike. Third, quantities are measured rather than inferred whenever a direct measurement exists. Fourth, novelty is stated as “not found in the literature consulted”, never as “does not exist”; one such claim in an early draft of this work had to be corrected when a 2024 paper measuring exactly the quantity in question was found, and the correction is left visible in the working notes.

Everything is released: the VHDL and its testbenches, the Python models and analysis, the firmware, the synthesis flow, the official test vectors used and the raw data of every campaign, with the commands that reproduce each table.

1.6 Structure of the document

Chapter 2 reviews ring-oscillator generators, their stochastic models, the standards and the commercial landscape. Chapter 3 builds the chain from transistor to min-entropy and validates every link. Chapter 4 describes the digital engine and Chapter 5 its verification. Chapter 6 synthesizes the engine on standard cells and states the path and cost of an integrated circuit. Chapter 7 reports the hardware campaign and the comparisons, and Chapter 8 concludes.

Chapter 2

State of the Art

The review is organized along the chain the thesis builds: how ring oscillators are turned into random bits, how their entropy is modeled and bounded, how their jitter is measured from inside the device, what the two governing standards demand, which symmetric primitives fit a constrained engine, what the commercial parts offer, and what changes when the design leaves the FPGA for silicon. The last section states the gap this work addresses, in the guarded form the working rules require.

2.1 Ring-oscillator TRNG architectures

A ring oscillator is an odd chain of inverters closed on itself; its period fluctuates because each transition is delayed by a random amount, and that fluctuation — jitter — is the noise source of an entire family of generators. What distinguishes the architectures is how the jitter is accumulated and sampled [8, 9].

The **elementary ring oscillator** (ERO) samples one ring with a flip-flop clocked by a second ring divided by K_D ; the divider sets the accumulation time. It has the most mature stochastic model [10, 11, 12] and an embedded jitter measurement [13], at the price of a very low output rate (K_D of 10^4 to 10^5 : 80 000 in Spartan-6 and 135 000 in Cyclone V in Petura’s implementation, about 430 000 in Fischer and Lubicz’s design for 0.997 bit of entropy). The **multi-ring** generator of Sunar, Martin and Stinson [14] XORs many rings sampled by a reference clock; Wold and Tan [15] added a flip-flop per ring before the XOR and reported 100 Mbit/s in under 100 logic elements. Its weakness was exposed by Bochard *et al.* [16]: part of the apparent randomness is *pseudo*-randomness from the interaction of rings at different frequencies, and the independence the proof assumes does not hold on an FPGA. **Coherent sampling** (COSO) uses two rings of nearly equal period and counts the beat [17, 18]; its model gives min-entropy directly from the distribution of the count [19], and dynamic calibration of reconfigurable rings makes it portable across devices and placements (3.30 Mbit/s in Spartan-6, 4.65 Mbit/s in Spartan-7) [20, 19]. The **transition-effect ring** (TERO) counts how many times a bistable structure oscillates

before settling [21], with a full physical-to-stochastic model [22]; the **self-timed ring** (STR) circulates several events in an asynchronous ring to obtain phases spaced by the order of the jitter [23], reaching 154 Mbit/s; and the **ES-TRNG** of Yang *et al.* uses the carry chain as a time-to-digital converter to encode the edge position with 10 LUTs and 5 flip-flops [24, 25].

Table 2.1 reproduces the comparison Petura *et al.* made of the AIS-compliant cores on one device [8]. The entropy column is a statistical estimate (the AIS 31 T8 test), not a model bound; the feasibility column is the authors’ qualitative note, where COSO and TERO score worst because they need manual adjustment per device. No comparable table exists for Artix-7; the open OpenTRNG project implements ERO, multi-ring and COSO on an Arty A7 with the same XC7A35T die as the Basys 3 but publishes neither entropy nor throughput figures [26].

Table 2.1: AIS-compatible TRNG cores on Xilinx Spartan-6 (45 nm), after Petura *et al.* [8], Table II. Entropy is the T8 estimate; “feasibility” is the authors’ rank (1 best).

Core	LUT/reg.	Power (mW)	Rate (Mbit/s)	Entropy/bit	Feasib.
ERO	46/19	2.16	0.0042	0.999	5
COSO	18/3	1.22	0.54	0.999	1
MURO	521/131	54.72	2.57	0.999	4
PLL	34/14	10.6	0.44	0.981	3
TERO	39/12	3.31	0.625	0.999	1
STR	346/256	65.9	154	0.998	2

2.2 Stochastic models and entropy bounds

The requirement that a physical generator come with a *stochastic model* — a mathematical description of the noise source from which the entropy of the output can be derived — was introduced by Killmann and Schindler [11, 27] and is mandatory in AIS 20/31. For ring oscillators the reference model is Baudet *et al.* [10]: the phase of the sampled ring is a Wiener process with drift μ and volatility σ^2 , the sampled bit is the half-period the phase falls in, and everything depends on the quality factor $Q = \sigma^2 \Delta t$ (accumulated phase variance between samples) and the frequency ratio $\nu = \mu \Delta t$. The model gives the bit probability conditioned on the previous phase, the probability of any output vector, and a lower bound on the Shannon entropy rate, $H \geq 1 - \frac{4}{\pi^2 \ln 2} e^{-4\pi^2 Q}$, whose exact status is examined in Chapter 3. Petura *et al.* rewrite it in design quantities, $H = 1 - \frac{4}{\pi^2 \ln 2} \exp(-\pi^2 \sigma_{th}^2 K T_2 / T_1^3)$, and Fischer and Lubicz invert it to obtain the divider K_D from a measured jitter [13]. Ma *et al.* give an alternative model used by the Leuven group [12]; Haddad *et al.* question the mutual independence of jitter realizations on which these cancellations rest [28]; and Saarinen warns that Baudet’s bound “is never lower

than 0.415 even when Q approaches zero” and is safe only under additional assumptions, proposing a bound valid over the whole range [29].

Two refinements matter for a design. First, not all jitter is entropy. Fischer and Lubicz separate the accumulated phase variance into a random-walk term, linear in time, produced by thermal (white) noise, and flicker ($1/f^\beta$) terms whose variance grows quadratically and dominate at long accumulation times [13]; deterministic, global jitter from the supply and substrate behaves like the latter and can be known or induced by an adversary [30]. Only the linear term may be credited, which is why a divider cannot be made arbitrarily large to compensate a weak source. Second, every embedded measurement is *differential*: two rings sharing a supply see the same global jitter, which cancels in their relative phase, leaving the sum of their local jitters. Lubicz and Skórski formalize this “jitter transfer principle” and show that with two rings one obtains only the sum, while three rings and three pairwise measurements recover the individual variances; their experiment also shows the assumption of variance linear in period failing by a factor of two in one case [31].

2.3 Embedded jitter measurement

Oscilloscopes cannot resolve the picoseconds of jitter of a ring inside a chip without an external divider [32], so the useful methods measure from within. The **counter method** counts periods of one ring in a fixed window of the other; the variance of the count grows with the accumulated relative jitter [11, 33], with a quantization floor of one count and contamination by flicker at long windows. The method of **Fischer and Lubicz** [13] needs no divider: in $K \approx 10^4$ blocks of $N \approx 100$ bits sampled at $K_D = 1$, the fraction of pairs (b_j, b_{j+M}) that differ tracks the phase accumulated over M periods, and its variance across blocks, plotted against M between 200 and 1600, gives the jitter per period with an error under 5% in simulation; on a Cyclone III they measured $\sigma = 5.01$ ps per 7.81 ns period, and showed that cooling the chip reduces σ — which makes the same measurement an online test. **Coherent sampling** yields the accumulated jitter from the variance of the beat count with no extra hardware [19]. Most recently Benea *et al.* characterize the jitter of rings with the **Allan variance**, fitting $\sigma_t^2(t) = a_0 + a_1t + a_2t^2$ to separate quantization, thermal and flicker terms, and do so both on an Artix-7 — the FPGA family of this work — and on a 28 nm FD-SOI circuit, with data and scripts released [32]. Published per-period jitter figures cluster at 2–4 ps for periods of 4–8 ns in Spartan-6 and Cyclone V [8], 5 ps in Cyclone III measured embedded [13], and 2–2.5 ps in Cyclone III and Virtex-5 for self-timed rings [23]; the Artix-7 coefficients of Benea *et al.* are published in counter units, not seconds, so an absolute figure for the XC7A35T remains to be measured.

2.4 Standards: AIS 20/31 v3.0 and NIST SP 800-90B

Two documents govern how a physical generator is assessed. The German **AIS 20/31**, in its version 3.0 of September 2024 [5], replaces the 2011 requirement of 0.997 bit of Shannon entropy per internal bit [34] with a choice in class PTG.2: Shannon entropy ≥ 0.9998 and/or min-entropy ≥ 0.98 , together with $P(1) \in (0.493, 0.507)$; the BSI’s transition policy allows new evaluations under the old version only until March 2026 [35]. Its requirements are structural as much as numerical: a verifiable stochastic model is mandatory; the online test must be derived from the model, failing with high probability when the source parameter leaves the admissible set; a total-failure test must detect entropy collapsing to zero, and the repetition-count test of SP 800-90B is explicitly accepted for it; and the black-box suite T_{irn} now includes two of NIST’s predictors (MultiMMC and LZ78Y), a deliberate harmonization [5, 36].

The American **SP 800-90B** [6] estimates *min-entropy* — the adversarial quantity, $-\log_2$ of the most probable value — from at least one million raw samples and a 1000×1000 restart matrix, using the most-common-value estimator if the source is shown to be i.i.d. and otherwise the minimum of ten estimators, four of them predictors that try to guess the next sample. It approves two continuous health tests and requires no others: the Repetition Count Test with cutoff $C = 1 + \lceil -\log_2 \alpha / H \rceil$ and the Adaptive Proportion Test over a window of 1024 samples for binary sources (512 for non-binary), $C = 1 + \text{CRITBINOM}(W, 2^{-H}, 1 - \alpha)$, with $\alpha = 2^{-20}$ recommended. It lists the conditioning components whose output may claim full entropy — HMAC, CMAC, CBC-MAC, approved hashes and the two derivation functions of SP 800-90A — and gives the formula for the entropy of a conditioned output as a function of input length, input entropy and internal width. The reference implementation of the estimators is public [37].

The two standards agree on what the older statistical suites cannot do. SP 800-22 [38] tests uniformity and independence of a bit stream; a deterministic generator with zero entropy passes it by construction, and so does a weak physical source behind a cryptographic post-processor [29]. NIST’s 2022 decision to revise the publication states the point in so many words, “rejecting its use for assessing cryptographic random number generators” [4]. The empirical record agrees: a certified smart-card generator showed consistent biases while passing most tests [2, 39], and the ESP32’s generator passes the same screenings with its radio off, when Espressif’s own documentation says it returns pseudo-random values [40, 41, 3]. The academic literature on that generator applies only SP 800-22 or Dieharder [42, 43]; no SP 800-90B characterization of it was found.

2.5 Symmetric cores for constrained hardware

The engine needs a block cipher and, if it uses one, a hash. For AES-128 (ten rounds [44]) the design space runs from fully unrolled pipelines for throughput to 8-bit datapaths for area: Good and Benaissa report 124 slices and two block RAMs at 2.2 Mbit/s on a Spartan-II at one extreme and 25 Gbit/s at the other [45]; Chodowiec and Gaj’s compact design uses 222 slices and three block RAMs [46]; and the smallest ASIC realizations reach 2.4 kGE with an 8-bit datapath at 226 cycles per block [47], following the composite-field S-box line of Satoh [48], Canright [49] and Boyar–Peralta [50]. The composite-field constructions optimize ASIC gate counts and lose their advantage in an FPGA LUT, where a direct table is both faster and simpler. Open reference designs with published Artix-7 figures exist: a one-round-per-cycle encryptor at 1104 LUTs and 294 MHz [51], and a 32-bit iterative encryptor/decryptor at 2298 slices and 97 MHz [52]. For an engine whose output leaves through an I2C link, any of these is far faster than the link, and the choice is dictated by verifiability and area, not speed.

SHA-256 [53] in its iterative form costs about 750 slices at 108 MHz on Artix-7 [54], comparable to a compact AES; its role in a generator would be as the vetted conditioner of SP 800-90B. The deterministic generators of SP 800-90A [55] — Hash_DRBG, HMAC_DRBG and CTR_DRBG — differ mainly in cost: CTR_DRBG with AES-128 keeps a state of 256 bits plus a counter and produces 128 bits per block encryption, Hash_DRBG needs 440-bit modular additions, and HMAC_DRBG two hash compressions per output block. Without a derivation function CTR_DRBG requires full-entropy input of exactly the seed length; with one, the usual configuration in practice, it accepts arbitrary input. Woodage and Shumow’s analysis warns that the standard’s flexibility permits choices that worsen state leakage and recommends updating after every generate and reseeding far below the 2^{48} limit [56]; SP 800-90C, final since 2025, frames how a physical source and a DRBG combine [57]. For authentication, AES-CMAC (SP 800-38B [58], RFC 4493 [59]) costs one block encryption per message block and is itself a vetted conditioner. The official vectors for all of these are public [60, 61, 62, 63].

2.6 Commercial secure elements

The product category this engine belongs to exists. A secure element is a small integrated circuit that generates and stores keys and performs symmetric and asymmetric operations for a host microcontroller, almost always over I2C. Four parts dominate the low-cost segment: Microchip’s ATECC608B, NXP’s SE050, Infineon’s OPTIGA Trust M and ST’s STSAFE-A110 [7, 64, 65, 66]. The ATECC608B, at 0.90 \$ in single units [67], offers AES-128, SHA-256/HMAC, ECDSA/ECDH on P-256, sixteen key slots and an I2C interface, and its entropy source holds a NIST validation under SP 800-90B, obtained precisely

with ring oscillators [68]. Two of its competitors make no reference to SP 800-90 in their datasheets; the third states that its generator is “designed to meet” the standard. The lesson for this work is where an academic design can and cannot compete: not on price, where a certified part costs less than a euro, but on the transparency of the entropy, which is exactly what a black-box part cannot offer and what the standards now demand.

2.7 From FPGA to ASIC: what changes in the entropy source

Digital logic ports from an FPGA to standard cells with a known penalty — Kuon and Rose measured $40\times$ in area, $3.2\times$ in delay and $12\times$ in dynamic power for logic-only designs in 90 nm, and $27\text{--}49\times$ in area for AES specifically [69]; the ring oscillator does not port at all, because in silicon it is built from inverter cells whose noise, variability and coupling differ from a LUT chain. Three published findings define what a silicon implementation must add. Benea *et al.* show that flicker noise, whose weight grows in newer nodes, changes the character of the jitter and, against expectation, that treating it as a legitimate source can raise throughput fourfold at equal entropy [32]. Yang, Blaauw and Sylvester show that process variation — larger in 40 nm than in 180 nm — must be absorbed by a configurable ring with a tuning loop for the generator to survive -40 to 120 °C and 0.6 to 0.9 V [70]. And two attacks show why a security device cannot ignore its environment: a frequency injected on the supply locks the rings and removed the jitter of a commercial secure microcontroller, reducing its key space from 2^{64} to 3300 [1], and a few hundred microwatts of radiated field biased a fifty-ring generator until it failed every test [71]; the published defence combines supply filtering with the same calibration loop [70], and device aging under bias-temperature instability and hot-carrier injection degrades poorly-timed generators over years while leaving well-timed ones intact.

Access to silicon is more open than it appears. The University of Sevilla and IMSE-CNM are members of Europractice [72], whose academic price list is public: a mini@sic block costs 3 430 € in UMC 180 nm and 3 691 € in TSMC 65 nm, and IHP’s SG13G2 is offered at 5 110 €/mm² with a free run for open-source designs under 2 mm² [73, 74, 75]. The institute’s hardware-security area has already taken ring-oscillator primitives from Xilinx 7-series FPGAs, evaluated with SP 800-22 and SP 800-90B, to characterized 65 nm silicon [76, 77, 78, 79], and has fabricated cryptographic circuits before [80]. The open standard-cell libraries that make a synthesis chapter possible — IHP SG13G2, SkyWater SKY130, GlobalFoundries GF180MCU and the predictive Nangate 45 nm — come with liberty, LEF and GDS views, and the literature compares such designs in gate equivalents normalized to the minimum NAND2 of the library [81, 82, 83, 84, 85], with the caution that logic-synthesis area is a lower bound that grows by a factor of 1.7 to 4.6 to routed

core area.

2.8 Gap addressed by this work

No absolute novelty is claimed. The ERO and its model, the embedded jitter measurement, the survey of AIS-compliant cores, the calibrated COSO on 7-series devices and an open implementation on the very die of the Basys 3 all exist, and the institute’s own group has crossed from 7-series FPGAs to 65 nm silicon with ring-oscillator primitives. What was *not found* in the literature consulted, and what this work sets out to provide in a modest but verifiable way, is the following.

1. A per-period jitter figure for the Artix-7 XC7A35T in absolute units, obtained inside the device with a differential method, with the thermal and flicker components separated and with stated uncertainty — the published Artix-7 coefficients being in counter units.
2. The complete chain *measured jitter* $\rightarrow$ *quality factor* $\rightarrow$ *min-entropy bound* $\rightarrow$ *divider* $\rightarrow$ *health-test cutoffs*, applied to one platform and confronted with the ten non-i.i.d. estimators of SP 800-90B and with the PTG.2 requirements of AIS 20/31 *version 3.0*, which most of the earlier literature could not use.
3. An SP 800-90B characterization of the ESP32 generator in its operating states, and a controlled comparison between a white-box FPGA source and a black-box commercial one under the same methodology.
4. The use of a commodity microcontroller with its own cryptographic accelerators as an independent oracle for an FPGA engine, over a real bus, with known-answer, interoperability and challenge-response tests.
5. A single-primitive key-generation engine — AES for conditioning, generation and authentication — written as portable RTL, verified against official vectors, synthesized on an open 130 nm library with the result stated as a lower bound, and costed for an academic shuttle.

Each item is formulated as “not found” rather than “does not exist”. One earlier claim of this kind in the working notes — that no Artix-7 jitter measurement had been published — had to be corrected when Benea *et al.*’s paper was found, and the correction is kept visible there.

Chapter 3

Entropy Source: from Transistor to Min-Entropy

This chapter builds the chain that the rest of the thesis rests on: from the physical jitter of a ring oscillator to a defensible figure of min-entropy per output bit, and from that figure to the design parameters of the sampler. Every step is validated before it is used. The statistical estimators are validated against synthetic data of known jitter; the analytic bounds are validated against an exact numerical computation; and the transistor-level bench is validated by measuring its own resolution before anything is measured with it. All figures in this chapter are [SIMULATED] unless stated otherwise; the hardware campaign is reported in Chapter 7.

3.1 The elementary ring oscillator

The entropy source is an Elementary Ring Oscillator (ERO): a free-running ring oscillator RO1 whose output is sampled by a flip-flop clocked by a second ring oscillator RO2 divided by an integer K_D [10, 13]. Between two samples the phase of RO1 accumulates jitter for K_D periods of RO2; when the accumulated jitter is comparable to the period of RO1, the sampled bit becomes unpredictable. The ERO was chosen over higher-throughput alternatives (multi-ring XOR, coherent sampling, transition-effect rings) for one reason: it is the architecture for which a stochastic model with a provable entropy bound exists and for which an embedded jitter measurement method has been published and validated [8]. Throughput is not a requirement here — the engine consumes entropy only to seed a deterministic generator — whereas a demonstrable bound is.

A design decision that follows from the platform is that *the board oscillator is never used as a reference*. The Basys 3 clock source is a MEMS oscillator with a fractional Phase-Locked Loop (PLL) whose datasheet specifies a cycle-to-cycle jitter of up to 95 ps [ESTIMATED] [86], one to two orders of magnitude above the per-period jitter of a ring

oscillator in a 28 nm process (2–4 ps [ESTIMATED] [8]). Sampling RO1 with that clock would measure the phase noise of the PLL, a global and potentially inducible component, rather than the thermal noise of the ring. RO2 therefore samples RO1, and the 100 MHz system clock only drives the synchronous logic.

3.2 Stochastic model and the quality factor

Following Baudet *et al.* [10], the phase $\varphi(t)$ of RO1, measured in periods, is modeled as a Wiener process with drift μ and volatility σ^2 . Between two samples separated by Δt the phase advances $\nu = \mu \Delta t$ periods on average and accumulates a variance

$$Q = \sigma^2 \Delta t, \quad (3.1)$$

the *quality factor*. The output bit is $s = 1$ when the fractional phase lies in $[1/2, 1)$. Conditioned on the previous phase φ_0 , the bit is Bernoulli with

$$P(s = 1 \mid \varphi_0) = \sum_{k \in \mathbb{Z}} \left[\Phi\left(\frac{k+1-\varphi_0-\nu}{\sqrt{Q}}\right) - \Phi\left(\frac{k+\frac{1}{2}-\varphi_0-\nu}{\sqrt{Q}}\right) \right], \quad (3.2)$$

where Φ is the standard normal distribution function; this is the wrapped Gaussian evaluated exactly. Its first harmonic gives the maximum bias $\varepsilon_{\max} \approx (2/\pi) e^{-2\pi^2 Q}$, from which Baudet’s Corollary 1 yields the well-known expression

$$H(s \mid \varphi_0) = 1 - \frac{4}{\pi^2 \ln 2} e^{-4\pi^2 Q} + O\left(e^{-6\pi^2 Q}\right). \quad (3.3)$$

3.3 Exact conditional entropy vs. the asymptotic bound

Equation (3.3) is routinely cited as a *lower bound* on entropy per bit. Read carefully, the inequality in the corollary is only that the entropy rate of the sequence is at least the entropy of one bit conditioned on the exact previous phase; the right-hand side is an asymptotic expansion *of* that conditional entropy, not a bound on it. Expanding the binary entropy in the bias, $h_2(\frac{1}{2} + \varepsilon) = 1 - \frac{1}{\ln 2} \sum_{n \geq 1} \frac{(2\varepsilon)^{2n}}{2n(2n-1)}$, and averaging over a uniform φ_0 with $\varepsilon = A \sin(2\pi\varphi)$, $A = (2/\pi)e^{-2\pi^2 Q}$, gives

$$H(s \mid \varphi_0) = 1 - \frac{4}{\pi^2 \ln 2} e^{-4\pi^2 Q} - \frac{8}{\pi^4 \ln 2} e^{-8\pi^2 Q} - \dots \quad (3.4)$$

All omitted terms are negative, so *any* truncation overestimates the entropy; the next term decays as $e^{-8\pi^2 Q}$, not $e^{-6\pi^2 Q}$. Table 3.1 compares the exact value computed from (3.2) on a grid of 8192 phases with the first- and second-order expansions. Adding the second-order term reduces the error of (3.3) by a factor of 95 at $Q = 0.10$ and 4970 at

$Q = 0.20$, which confirms the derivation.

Table 3.1: Conditional Shannon entropy per bit: exact wrapped-Gaussian computation vs. truncated expansions [SIMULATED].

Q	Exact	Order 2	Order 1 (Eq. 3.3)	Error of order 1
0.02	0.7013194	0.7100952	0.7345213	$+3.3 \times 10^{-2}$
0.05	0.9163001	0.9164920	0.9187783	$+2.5 \times 10^{-3}$
0.10	0.9886728	0.9886733	0.9887174	$+4.5 \times 10^{-5}$
0.15	0.9984319	0.9984319	0.9984327	$+8.5 \times 10^{-7}$
0.20	0.9997823	0.9997823	0.9997823	$+1.6 \times 10^{-8}$

In the operating region ($Q \approx 0.2$) the discrepancy is 10^{-8} and irrelevant; for an under-dimensioned generator ($Q \approx 0.02$) the truncated formula credits 3.3 % of entropy that does not exist. This thesis therefore uses the exact computation as the conservative value and cites (3.3) as what it is: an excellent asymptotic approximation for $Q \gtrsim 0.1$. The same numerical machinery shows that the min-entropy obtained from the first-harmonic bias, $H_\infty = -\log_2(\frac{1}{2} + \varepsilon_{\max})$, agrees with the exact worst-case value to better than 10^{-4} bit over $Q \in [0.05, 0.30]$.

3.3.1 Design target

AIS 20/31 version 3.0 [5] admits two alternative criteria for class PTG.2: Shannon entropy ≥ 0.9998 or min-entropy ≥ 0.98 per internal bit. Solving both by bisection on the exact entropies gives $Q \geq 0.2022$ and $Q \geq 0.2286$ respectively. The min-entropy criterion is the more demanding one and is also the quantity NIST SP 800-90B works with, so it is adopted here: **the design target is $Q \geq 0.23$, with a set point of $Q = 0.30$** ($H_\infty = 0.9951$), the margin covering the reduction of thermal jitter at low temperature, which is the standard adversarial test of Fischer and Lubicz [13].

3.4 Embedded jitter estimator and its two corrections

To choose K_D one must know σ^2 , and the method of Fischer and Lubicz [13] measures it from the bits themselves: with $K_D = 1$, the fraction c of pairs (b_j, b_{j+M}) that differ within a block of N pairs estimates the phase accumulated over M samples, and its variance across K blocks grows as $4MQ_{\text{raw}}$. Implemented literally, with a straight-line fit, the estimator *underestimates* Q_{raw} by 13 % to 38 % depending on the jitter level. Two causes were identified on synthetic data with known Q and corrected:

1. **Vertex effect.** c is not linear in the accumulated phase; it is a *triangular* wave of period one and slope ± 2 . When the deterministic phase $M\nu \bmod 1$ falls near

a vertex the distribution straddles it and the variance collapses — to 0.38 of its nominal value in one measured case. The vertex is detected from the measured mean itself ($\bar{c} \approx 0$ or 1 is a vertex, $\bar{c} \approx \frac{1}{2}$ the middle of the ramp), so distances M with $\bar{c}$ outside $[0.15, 0.85]$, or for which $3\sqrt{MQ}$ reaches the vertex, are discarded iteratively.

2. **Non-constant estimation noise.** Estimating a fraction from N pairs adds a variance that scales with $\bar{c}(1 - \bar{c})$, which changes with M . Fitting it as a constant intercept biases the slope; it is fitted instead as a second regressor, $\text{Var}(c) = Q(4M) + \kappa \bar{c}(1 - \bar{c})/N$.

After both corrections the estimator recovers the injected Q_{raw} within $\pm 2\%$ over a range of $150\times$ (Table 3.2), with a bias of -0.6% and a dispersion of 4.0% over eight independent seeds for $K = 20\,000$ blocks. The fitted accumulation exponent is 1.009 for a simulator without flicker noise; measured on hardware, the same exponent separates the thermal component ($b = 1$, which counts as entropy) from flicker and global jitter ($b = 2$, which does not).

Table 3.2: Recovery of a known Q_{raw} by the corrected embedded estimator ($K = 20\,000$ blocks of $N = 100$ pairs) [SIMULATED].

Q_{raw} injected	Estimated	Error	M values used
2.0×10^{-7}	1.979×10^{-7}	-1.07%	19
5.0×10^{-7}	5.060×10^{-7}	$+1.21\%$	18
2.0×10^{-6}	2.037×10^{-6}	$+1.86\%$	16
8.0×10^{-6}	8.039×10^{-6}	$+0.48\%$	9
3.0×10^{-5}	2.977×10^{-5}	-0.78%	5

A practical consequence for the hardware is that the blocks need not be contiguous: each requires only $N + M$ consecutive bits, and independent blocks give a slightly better estimate (-0.30% error with 20 000 independent chunks). The capture buffer in the FPGA therefore holds a single chunk of a few kilobits, filled at ring speed and read out slowly, instead of megabits of contiguous samples.

3.5 Transistor-level bench and its resolution

The entropy source is the only block of the engine tied to the technology, and in an integrated circuit it is designed at transistor level. A SPICE bench was built around a five-stage ring with an enabling NAND gate, with every technology-dependent parameter (device models, widths, minimum length, supply) confined to a header so that moving from the generic placeholder models used here to a foundry Process Design Kit (PDK) changes one block of the netlist. Jitter is extracted from the threshold-crossing instants

of the output by the second difference $D(M) = t_{k+2M} - 2t_{k+M} + t_k$, i.e. the Allan variance applied to phase data, whose expectation $2M\sigma^2$ is free of the bias that a simple difference over overlapping windows carries (measured at 5%), and which is the method used by Benea *et al.* to characterize rings both in Artix-7 and in 28 nm silicon [32]. Estimates at different M are combined with inverse-variance weights proportional to the number of independent windows; without that weighting the dispersion stayed at 15% regardless of the simulated length.

The bench measures its own resolution first: with all noise sources disabled, the residual jitter is the numerical noise of the simulator. The first attempt gave 0.33 ps, of the same order as the jitter to be measured; it was entirely the start-up transient of the oscillator, and discarding the first twenty cycles brings the floor to **1.38 fs**, three orders of magnitude below the signal. Only then was noise injected. Jitter scales with the square root of the injected spectral density ($\times 2.01$ measured for $\times 4$ in density, against $\times 2.00$ expected) up to the point where the r.m.s. noise current reaches about 80% of the stage drive current; there the small-signal regime breaks down, the mean period shifts, and the scaling law fails by a factor of five. A validity criterion follows for every campaign, including those on a real PDK: keep injected noise below $\sim 40\%$ of the stage current and verify that the mean period does not move when noise is enabled. With the placeholder models the ring oscillates at 4.3 GHz, which is not physical; the numbers that enter the thesis come from the foundry models, where transient noise analysis injects the device noise itself.

3.6 Design target and divider selection

With σ/T measured per period, $Q_{\text{raw}} = (\sigma/T)^2$ and the divider follows from $K_D = \lceil Q_{\text{target}}/Q_{\text{raw}} \rceil$. For the range of jitter expected in a 28 nm process the divider lies between 3×10^4 and 5×10^5 and the throughput per ring between 1 and 20 kbit/s. The model reproduces the published design point of Fischer and Lubicz ($K_D \approx 430\,000$ for $\sigma = 5.01$ ps over $T = 7.81$ ns) [13]. Such rates are sufficient here: seeding the deterministic generator with 256 bits of full entropy requires about 512 raw bits at $H_\infty \approx 0.98$, i.e. between 0.03 and 0.5 s per seed, and the engine issues a seed only at start-up and at reseed intervals. If more throughput were ever needed, the clean route is several independent EROs, each with its own bound, rather than a multi-ring XOR whose independence cannot be demonstrated [16].

Chapter 4

Digital Engine

The digital engine turns raw bits from the entropy source into keys and uses them, and it is written so that the same Register-Transfer Level (RTL) runs on the FPGA used for verification and, later, on standard cells. Nothing in it is tied to a vendor: no primitives, no memory macros, no clock-management blocks. The only technology-dependent block of the whole design is the pair of ring oscillators of Chapter 3.

4.1 Architecture and the single-primitive decision

Figure 4.1 summarizes the data path. Raw bits leave the ERO in the clock domain of the sampling ring, cross to the system clock one bit at a time, and feed three consumers: the health tests, the seed collector of the deterministic generator, and a capture buffer used only for characterization. Keys come out of a Counter-mode Deterministic Random Bit Generator (CTR_DRBG); the current key is loaded into an AES-CMAC block that answers challenges; and an Inter-Integrated Circuit (I2C) slave exposes everything as a register map.

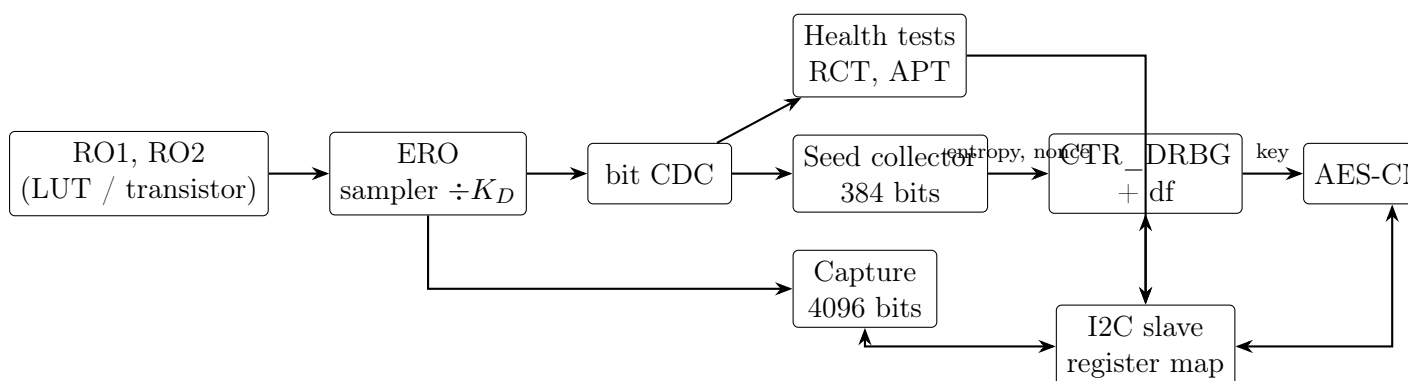


Figure 4.1: Data path of the engine. Only the ring oscillators are technology dependent; every other block is portable RTL.

The decision that shapes the whole engine is to use **one cryptographic primitive**,

AES-128, for the three jobs: conditioning the entropy, generating keys and authenticating. The first architecture had a SHA-256 core as conditioner; the synthesis estimates of Chapter 6 showed that it would take almost half the digital area, and SP 800-90B lists AES-based constructions — CMAC, CBC-MAC and the block-cipher derivation function of SP 800-90A — among its *vetted* conditioning components [6]. Dropping the hash brings the engine from an estimated ~ 25 to ~ 13 Gate Equivalents (GEs) (thousands), at the price of moving from a CTR_DRBG *without* derivation function, which needs full-entropy input, to one *with* it, which is the common configuration in practice and accepts non-full-entropy input directly. The AES core is encryption-only: counter mode and CMAC never decrypt, and offering decryption as a service to the ESP32 would be pointless since the ESP32 has its own accelerator. Cross-verification is unaffected: the engine encrypts and the ESP32 decrypts with mbedtls.

4.2 AES-128 encryption core

The core is iterative, one round per clock cycle, eleven cycles per block (initial key addition plus ten rounds), with the round keys expanded on the fly from the previous one — forward expansion is all that encryption needs, and it saves storing eleven round keys. At 100 MHz that is 1.16 Gbit/s of core throughput, far above what any I2C link can carry, so pipelining would be wasted area; the iterative form is chosen instead because one round is a pure combinational function that can be checked directly against the standard [44]. The S-box is a 256-entry table synthesized as logic, MixColumns is written as the usual `xtime` network, and the state follows the column-major convention of FIPS 197 with byte 0 on the left, so that hexadecimal strings in the standard map one-to-one onto the VHDL vectors.

The core was verified against the five official vectors available for AES-128 encryption (FIPS 197 appendices B and C.1, and the three ECB blocks of SP 800-38A F.1.1) and against a chain of one thousand encryptions to expose any state left over between blocks. A check that had not been planned turned out to be the most convincing: the encryption of the all-zero block under the SP 800-38A example key gave `7df76b0c...546f`, which is exactly the intermediate value L published by NIST in its CMAC examples [87].

4.3 AES-CMAC

AES-CMAC [58, 59] is the primitive of the challenge-response: the ESP32 sends a nonce, the engine returns its tag under the current key. The block computes the sub-keys K_1 , K_2 once when a key is loaded (one encryption of the zero block and two conditional left shifts with the polynomial constant `0x87`) and then runs CBC-MAC over the message, applying K_1 to a complete last block or padding with `0x80` and applying K_2 otherwise.

It was chosen over HMAC-SHA-256 because it reuses the only primitive and costs one encryption per block against two hash compressions. The block passes the four vectors of RFC 4493 (empty message, one block, 40 and 64 bytes), which exercise both padding paths.

4.4 CTR_DRBG with derivation function

The generator implements CTR_DRBG with AES-128 and the block-cipher derivation function of SP 800-90A [55]. Its input format is fixed on purpose: 256 bits of raw entropy and 128 bits of nonce, so that $S = L \parallel N \parallel E \parallel \text{nonce} \parallel 0x80 \parallel 0 \dots$ is exactly four blocks and the derivation function becomes a closed sequence of fourteen encryptions; a variable-length input would cost a byte-oriented datapath for no benefit. Instantiation (or reseed) costs 14 AES operations, 183 cycles; generating 256 bits costs four, 53 cycles. The state (Key, V) is never exposed; the reseed counter is readable so that the host can enforce a reseed policy far below the 2^{48} limit of the standard.

Verification proceeded in two steps. First, a bit-exact reference model was written in Python — with its own AES, independent from the VHDL — and run against the CAVP vectors: 960 AES-128 cases with and without derivation function, with and without reseed, all passing. Second, that model generated the expected outputs for the fixed input layout of the hardware, and the VHDL reproduced them: two consecutive generations, a reseed followed by a generation, the reseed counter, and the fact that a new instantiation erases the previous state.

4.5 Health tests

The two tests approved by SP 800-90B run continuously on the raw bits [6]. The Repetition Count Test fails when $C_{\text{RCT}} = 1 + \lceil -\log_2 \alpha / H \rceil$ identical samples arrive in a row; the Adaptive Proportion Test counts, in a window of $W = 1024$ samples (the binary-source value; 512 is for non-binary sources), how many repeat the first one, and fails at $C_{\text{APT}} = 1 + \text{CRITBINOM}(W, 2^{-H}, 1 - \alpha)$. For the design target $H = 0.98$ and $\alpha = 2^{-20}$ these are $C_{\text{RCT}} = 22$ and $C_{\text{APT}} = 596$; both are generics, because H is set from the *measured* min-entropy, not assumed. The proportion test also checks the complement of the count, which the standard permits for binary sources: a source stuck at the value opposite to the window’s first sample would otherwise score zero and pass. Alarms are sticky until explicitly cleared, and while any alarm is raised the sequencer refuses to seed.

The testbench injects the failure modes the tests exist for: a healthy source passes twenty windows (worst run 13, worst proportion 540); a stuck source trips the repetition test at exactly sample 22; a source biased to 70 % trips the proportion test at 713 without tripping the repetition test; and a source stuck at the complement is caught. The first

“healthy” source of the bench — a shift register seeded with a single one — was itself caught by the tests, with a start-up run of 27 zeros and a first window at 76 %. The detector was right and the bench was wrong; the source now starts from a dense seed and is warmed up.

4.6 Capture path and clock-domain crossings

Raw bits are produced in the clock domain of RO2 and consumed in the system domain. The bit-serial path to the health tests and the seed collector uses a toggle synchronizer: the source registers the bit and inverts a flag; the destination synchronizes the flag through three flip-flops and reads the bit on the flag’s edge. It requires each bit to last at least eight system cycles, i.e. $K_D \geq 64$ for a ~ 700 MHz sampling ring at 100 MHz, which the generator mode satisfies by orders of magnitude. The characterization path does not cross bit by bit: the capture buffer is written in the ring domain at full rate into a 4096-bit memory and read out in the system domain afterwards, with the two directions of the handshake carried by toggle flags. Writing and reading never overlap, so no asynchronous FIFO is needed, and the buffer size follows from Section 3.4: one chunk of $N + M_{\max}$ bits is all the estimator needs.

4.7 I2C register interface and command sequencer

The product interface is I2C, as in commercial secure elements: two pins, addressable, and the bus the ESP32 uses for its peripherals. The slave implements the register-pointer protocol with auto-increment, a five-sample glitch filter on both lines as the specification requires, and open-drain outputs. It does not stretch the clock: at 400 kHz there are 250 system cycles per bit, plenty for any register access, and long operations (seeding, generation) are issued as commands and polled through a status register, which is how commercial devices behave. The slave was tested with a bus-functional master; two real defects surfaced — the pointer did not auto-increment on multi-byte writes, and the most significant bit of each read byte was driven one clock edge late, so every read would have been shifted — and were fixed.

The register map (Appendix A) exposes identification, status and control, the divider and capture-page configuration, the health-test diagnostics, the reseed counter, the challenge and its tag, a 256-bit random-output register and, *only while a test pin is asserted*, the current key and the capture buffer. The random-output register is filled by its own command with a separate generation of the DRBG, never by the one that produced the key: an earlier version of the design left the 256 bits of the key-generating call in that register, readable without the test pin, and thus leaked the key. The review that caught it is described in Chapter 5. In the FPGA the test pin is a switch; in an integrated

circuit it would be a fuse blown at production, because exposing either the key or the raw entropy is a vulnerability. A small sequencer serializes the commands: seeding collects 384 raw bits, aborts if a health alarm rises meanwhile, and instantiates or reseeds the generator; generation takes 256 bits, keeps the first 128 as the key, discards the rest and reloads the CMAC sub-keys; a separate command fills the random-output register from an independent generation; authentication runs the CMAC over the challenge block.

The whole engine was exercised as the ESP32 will exercise it: a bus-functional I2C master reads the identification, configures the divider, seeds, generates, reads the key in test mode, writes two challenges and reads their tags, generates again, captures raw bits and verifies that the key is unreadable without the test pin — sixteen checks. What a VHDL bench cannot know is whether the tags are *correct*, because the key came from ring-oscillator noise; the bench dumps key, challenges and tags to a file and a Python script recomputes the CMACs with the independent AES. They match. That is the same cross-check the ESP32 performs on hardware with mbedtls.

Chapter 5

Verification

5.1 Strategy: vectors, simulation, independent verifier

A cryptographic engine that is verified only against itself proves nothing: an error in the S-box would be reproduced identically in the model used to check it. The verification of this work therefore rests on three legs that do not share an implementation.

1. **Official vectors.** Every primitive is checked against the test vectors published with its standard: FIPS 197 and SP 800-38A for AES, RFC 4493 and the NIST example file for CMAC, and the CAVP response files for CTR_DRBG (with and without derivation function, with and without reseed). These are known-answer tests and they settle correctness of the algorithm.
2. **Independent reference models.** A bit-exact model of the generator was written in Python with its own AES, derived from the standard without looking at the VHDL. It is validated against the CAVP vectors first, and only then used to generate expected values for the fixed input layout of the hardware and to recompute the tags of the system-level bench. Two implementations that agree are worth more than one that passes.
3. **An independent verifier in the loop.** On hardware the ESP32 recomputes with mbedtls, and with its own AES accelerator, every result the engine produces, so that the final check is performed by a third, unrelated implementation over a real bus.

Two working rules apply throughout, both inherited from the author’s previous experimental work: the output of a compilation, simulation or measurement is never filtered before being read in full — a filter hides exactly the warning that mattered — and failures are recorded whole, never discarded. Twice in this work a filter on the simulator’s log hid a result, and both times the fix was to remove the filter.

5.2 Reference models in Python

`ctr_drbg_ref.py` implements AES-128 encryption, the block-cipher derivation function, CBC-MAC and CTR_DRBG with and without derivation function, in pure Python and from the text of SP 800-90A. The AES asserts the two FIPS 197 vectors at import; the DRBG is exercised against the CAVP response files following the CAVS procedure (instantiate, optional reseed, two generate calls, compare the second): **960 AES-128 cases, all passing**. One defect was found in the file parser rather than in the model — a section header must close the pending test case before the parameters change, otherwise the last case of every parameter block is evaluated with the next block’s parameters — and it is worth recording because its symptom, a single failure at count 14 of each block, looked exactly like an algorithmic error.

A second script, `gen_drbg_vectores.py`, drives the validated model through the sequence the hardware supports (instantiate with 32 bytes of entropy and 16 of nonce, generate twice, reseed, generate) and emits a VHDL package of expected values; if the input layout of the hardware ever changes, that script is the only place to change it. A third, `verifica_top.py`, recomputes AES-CMAC from the same AES and validates itself against RFC 4493 before checking anything.

5.3 Testbenches and results

All VHDL is simulated with GHDL 6 under a single runner that analyses the sources in dependency order and executes every bench, printing the simulator’s output unfiltered. Table 5.1 summarizes the seven benches and their state at the time of writing.

Table 5.1: Testbenches, what they check and their result (GHDL 6.0, VHDL-2008).

Bench	Checks	F
Ring oscillator	start/stop, $f = 1/(2Nt_{pd})$, zero floor, $\sigma_T = \sqrt{2N} \sigma_{stage}$	1
I2C slave	addressing, foreign address, multi-byte write, repeated-start read	1
AES-128 core	FIPS 197 B, C.1; SP 800-38A F.1.1 (3 blocks); 1000-block chain	1
AES-CMAC	RFC 4493 examples 1–4; repeatability	1
CTR_DRBG	instantiate, 2×generate, reseed, counter, re-instantiate, refuse unseeded	1
Health tests	healthy source, stuck, 70 % biased, stuck-at-complement, sticky/clear	1
Whole engine	ESP32 sequence over I2C: id, seed, key, 2 challenges, 2nd key, capture, key hidden	1

Some of what the benches found deserves to be stated, because a verification chapter that reports only passes has not verified much.

Ring model. The LUT primitives used to simulate the ring were first modeled with a strict unknown-propagation rule: any unknown input gave an unknown output. At the

start of a simulation every signal is unknown, so the ring never left that state and never started. A real NAND with its enable input at zero outputs one regardless of the other input; the model now evaluates every combination compatible with the known inputs and resolves the output when they all agree. Without this the entire system-level bench would have been impossible.

I2C slave. Two genuine design errors: the register pointer did not advance on multi-byte writes, so consecutive bytes overwrote the same address; and on reads the most significant bit was driven one clock edge too late, shifting every byte. The first was fixed by incrementing the pointer one state after the write strobe, so the address is still valid during the strobe; the second by loading the shift register and driving its first bit on the same falling edge that releases the acknowledge. A third defect was in the bench itself: initializing the register memory from the stimulus process created a second driver on the same signal and every conflicting bit resolved to unknown.

Health tests. The “healthy” source of the first bench, a shift register seeded with a single one, started with a run of 27 zeros and a first window at 76 %; the detector fired. The detector was right.

Key leak through the random register. The most serious finding did not come from a bench but from reading the design again before handing it over. The command that generated the key wrote the full 256-bit output of the DRBG into the random-output register, which is readable without the test pin; the first 128 of those bits *were* the key. The system bench even asserted that equality as a “check”. The fix separates the two: key generation leaves nothing readable, and a new command fills the random register from an independent generation, after the DRBG has updated its state. The bench now asserts the opposite, that the register contains the key in neither half, and the Python cross-check does the same.

Whole engine. With the rings at their physical speed the system bench, which drives almost a millisecond of I2C traffic, did not finish in twenty minutes of CPU: every ring transition costs two random draws in the jitter model. In that bench alone the stage delay is set ten times larger, with the same relative jitter, and this is stated in the bench. The sixteen checks pass, and the key, the two challenges and their tags are written to a file that `verifica_top.py` checks: both tags equal the CMAC recomputed by the independent Python AES, the readable random-output register contains the key in neither of its halves, and two generations give different keys. This is the first point at which a key born from ring-oscillator noise, conditioned, generated and used to authenticate, is confirmed by an implementation that shares nothing with the VHDL.

5.4 ESP32 verifier firmware

The firmware is an ESP-IDF 5 application for the classic ESP32 that talks to the engine through the register map of Appendix A and offers a console over the UART. Its commands map onto the verification plan: `id` reads identification and status; `sembrar` and `generar` issue the long commands and poll the busy flag; in test mode `generar` also reads back the key (provisioning) and checks that it equals the first sixteen bytes of the generated block; `reto N` performs N challenge-response rounds with nonces from `esp_random()`, recomputing each tag with `mbedtls_cipher_cmac` and counting matches and mismatches, the mismatches being printed in full; `crudo N` dumps N chunks of raw bits and `esp N` dumps blocks of the ESP32’s own generator, both in the framed, CRC32-protected format used in the author’s previous work so that the same statistical pipeline processes the two sources. CMAC is not enabled by default in the ESP-IDF build of `mbedtls` and is switched on in the project defaults; the console runs at 115 200 baud, the only rate that proved reliable with the board’s USB bridge in earlier campaigns, and the log level is set to errors only so that log lines cannot corrupt the frames. The firmware builds with ESP-IDF 5.3.2 into a 255 kB image; running it awaits the hardware.

5.5 FPGA bring-up

Pending hardware. The constraints the rings need are known from the literature and from the vendor documentation — `DONT_TOUCH` on the cells and nets of each ring (unlike `KEEP`, it survives implementation), `ALLOW_COMBINATORIAL_LOOPS` on one net of each loop, `set_disable_timing` to break the loop for static timing analysis, and one placement block per ring so that identical rings are neither merged nor placed side by side — and the first bitstream has a single purpose: to read the frequency counters and confirm that every ring oscillates. Nothing about entropy is claimed before that.

Chapter 6

Path to an Integrated Circuit

The engine is designed to end up as a dedicated integrated circuit that serves keys to a microcontroller, and the FPGA is its verification vehicle, not its destination. This chapter takes the same VHDL that passes the testbenches of Chapter 5, synthesizes it on an open standard-cell library, and states what a fabrication run would cost and what would have to change in the entropy source. A tape-out is explicitly out of scope of this thesis — not for cost or area, as will be seen, but for schedule — and everything here is labelled as what it is: a lower bound on area from logic synthesis, with the growth factors to core area taken from published data.

6.1 Standard-cell synthesis flow

The flow is deliberately reproducible with free tools. Yosys reads the VHDL through the GHDL front end, synthesizes each block flattened, maps flip-flops and logic onto the liberty file of the target library and reports cell count and area. The library is the open PDK of IHP, SG13G2, a 130 nm BiCMOS process with a maintained standard-cell library, compiled SRAM macros, I/O cells and OpenROAD support, chosen because it is the one open PDK that is alive, European and reachable through the same Europractice membership the university already holds. The gate equivalent is read from the liberty file itself rather than from memory: 1 GE = area of `sg13g2_nand2_1` = $7.2576 \mu\text{m}^2$. No clock constraint is applied: Yosys does not use one for area, so the figures do not depend on a target frequency. The ring oscillators are not synthesized; they are the one block designed at transistor level (Chapter 3).

6.2 Area per block in gate equivalents

Table 6.1 gives the result for each block. The digital engine — everything except the capture buffer — is about **62 kGE**, and AES is 51 % of it: the authenticator and the

generator each instantiate their own encryption core, 15.8 kGE apiece. The capture buffer is reported for completeness but must not be read as part of the product: its 46.7 kGE are a 4096-bit memory mapped onto flip-flops because the flow has no SRAM macros. In the FPGA it occupies a block RAM at no logic cost; in silicon it would be a compiled SRAM or, more sensibly, a much smaller buffer, because it is a test path that is disabled at production.

Table 6.1: Logic synthesis on IHP SG13G2, flattened, no routing. 1 GE = $7.2576 \mu\text{m}^2$. Lower bound on area [SIMULATED].

Block	Cells	Flip-flops	Area (μm^2)	kGE
AES-128 encryption core	10 439	262	114 771	15.81
AES-CMAC (with its AES)	13 210	909	169 766	23.39
CTR_DRBG with df (with its AES)	17 953	2 355	267 262	36.83
Health tests (RCT, APT)	568	85	8 986	1.24
ERO sampler and divider	117	24	2 078	0.29
Bit clock-domain crossing	11	7	408	0.06
I2C slave	310	63	5 507	0.76
Capture buffer (4096-bit RAM as flip-flops)	12 250	4 140	339 057	46.72

The numbers are internally consistent, and that consistency is how an error in the flow was caught: before flattening, the report returned for the authenticator and for the generator the area of their AES sub-module alone. An authenticator weighing exactly as much as a bare AES is impossible, so the report, not the design, was wrong. After flattening, the authenticator adds to the AES 647 flip-flops (sub-keys, chaining state, key) and 7.6 kGE, and the generator adds 2093 flip-flops and 21 kGE, about half of which are its registers.

6.3 Optimizations and their expected savings

The synthesis is naive on purpose — the RTL was written to be verifiable, not small — and it points at three optimizations, in order of return on effort:

1. **One shared AES core.** The top-level sequencer already serializes commands: the generator and the authenticator never run at the same time. Sharing the core saves ≈ 15.8 kGE, a quarter of the engine, for a trivial arbiter.
2. **Compact S-box datapath.** Twenty independent S-box tables synthesize to 15.8 kGE for the whole core; area-optimized designs in the literature range from 2.4 kGE for an 8-bit datapath at 226 cycles per block [47] to about 5 kGE for 32-bit datapaths. Speed is irrelevant here, so a 32-bit datapath with four shared S-boxes is the sensible point; estimated saving, 10 kGE per core.

3. **Fewer registers in the generator.** Of its 2355 flip-flops, the captured entropy and nonce (384), the seed material (256), the output (256) and the temporaries of the derivation function can be reduced by consuming the seed as a stream and sharing temporaries; roughly 800 flip-flops, about 4 kGE.

With the three, the digital engine falls to ≈ 20 kGE, which is the order of magnitude the state of the art anticipated (13 to 25 kGE for engines of this kind) and which, at the published growth factor of ≈ 2.35 from cell area to core area for an AES on this same library, is about 0.34 mm^2 of core. Each of these savings is stated as an estimate; measuring them by synthesizing the optimized variants is listed as future work, not claimed.

6.4 What the ring oscillator needs in silicon

The digital blocks port with the standard flow. The entropy source does not, and the literature that has done the crossing — notably a 2024 study that characterizes the same ring model on an Artix-7 and on a 28 nm FD-SOI circuit [32] — makes three points that the chip design must absorb.

First, the ring is rebuilt from inverter cells rather than look-up tables, and its jitter changes in value and in character: flicker noise gains weight in newer nodes, and the study finds, against expectation, that counting it as a legitimate source can raise throughput for the same entropy. The stochastic model and the estimator of Chapter 3 carry over unchanged; the measured σ does not. Second, process variation makes every die different, so the frequency and jitter of each ring must be calibrated per chip; the published all-digital designs that survive voltage and temperature sweeps do it with a configurable ring and a tuning loop [70]. Third, and most important for a security device, injecting a frequency on the supply locks the rings and removes the jitter: a 500 mV signal on the rail reduced the key space of a commercial secure microcontroller from 2^{64} to about 3300 [1], and a few hundred microwatts of radiated field destroyed the statistics of a fifty-ring generator [71]. The same calibration loop, together with supply filtering, is the published defence, and the health tests of Section 4.5 are the detector of last resort. None of this is in the FPGA prototype; it is the work that a silicon implementation adds.

6.5 Cost and schedule of an academic shuttle

Fabrication is more accessible than the size of the word suggests. The University of Sevilla and IMSE-CNM are members of Europractice, whose academic price list is public: a mini@sic block in UMC 180 nm costs 3 430 €, one in TSMC 65 nm 3 691 €, and IHP SG13G2 is offered at 5 110 €/mm² with a free multi-project run for open-source designs under 2 mm^2 [73]. An engine of $0.3\text{--}0.4 \text{ mm}^2$ fits many times over in the smallest block

of any of them; the chip would be limited by its pads, not its logic. The institute's own hardware-security group has already taken ring-oscillator primitives from Xilinx 7-series FPGAs to characterized 65 nm silicon, which makes the continuation of this work a matter of joining an existing line rather than opening one.

What does not fit in a master's thesis is the schedule: from tape-out to packaged samples takes months to over a year, the price does not include packaging, and the characterization of the returned dies is a project in itself. For comparison, a commercial secure element such as the ATECC608B costs 0.90 \$ in single units and ships with a NIST entropy source validation [7]; two of its direct competitors do not mention SP 800-90 anywhere in their datasheets. The honest position of this work is therefore not to compete on price but on *transparency of the entropy*: every bit of a key produced by this engine can be traced to a measured jitter and a stated bound, which is precisely what a black-box part cannot offer.

Chapter 7

Results and Comparison

This chapter collects what the work establishes and separates it, without softening, from what it does not. The engine has been modelled, written, verified in simulation against official vectors and synthesized on a real standard-cell library; it has not yet run on the Field-Programmable Gate Array (FPGA). Every number below therefore carries its provenance tag, and the entropy campaign — the one result that requires silicon on the bench — is stated as a protocol to execute, not as data.

7.1 Status against the objectives

Table 7.1 maps the objectives of Section 1.3 onto evidence.

The minimum defensible result declared in Chapter 1 — O1–O3 together with O5 — is reached on its modelling and design side. What O5 still lacks is the bring-up, not the design.

7.2 Transistor-level noise of the ring

The ring was simulated with a current noise source of controlled spectral density injected at the switching node, sweeping the density and measuring the standard deviation of the rising-edge crossings over 500 periods. Table 7.2 gives the result for a five-stage ring with a 10 fF load.

Three readings, and the third is the one that matters.

The numerical floor is far enough below the signal. With the noise source switched off the measurement returns 1.38 fs, which is the resolution of the simulation itself, not physics. It sits about 870 times below the smallest jitter to be measured, so the floor does not contaminate the sweep. Reaching it required care: the first attempt returned 0.33 ps, of the same order as the quantity under study, and the cause was the ring’s start-up

Table 7.1: Objectives and the evidence supporting each.

Obj.	State	Evidence
O1	met	Ring described at transistor level; noise swept in SPICE (Section 7.2)
O2	met	Jitter extracted from the waveforms; estimator validated on synthetic data with known Q
O3	met	Stochastic model closed: exact wrapped-Gaussian computation, design point $Q \geq 0.2286$ (Section 7.3)
O4	met	Engine written in portable VHDL-2008; seven testbenches pass against FIPS-197, SP 800-38B, RFC 4493 and Cryptographic Algorithm Validation Program (CAVP) vectors
O5	partial	Verifier firmware compiles and the protocol is closed; the board-level run is pending
O6	pending	Requires the FPGA on the bench (Section 7.7)
O7	met	Synthesis on IHP SG13G2, per-block area and optimization path (Chapter 6)
O8	partial	Structural comparison closed below; the numerical comparison against the ESP32 needs O6

Table 7.2: Period jitter of the five-stage ring against injected noise density [SIMULATED]. $\sigma_{\text{rel}} = \sigma/T$.

Noise density (A^2/Hz)	Crossings	T (ps)	σ (fs)	σ_{rel}
none (numerical floor)	500	230.30	1.38	6.0×10^{-6}
1×10^{-20}	501	230.37	1200	5.2×10^{-3}
4×10^{-20}	507	229.98	2416	1.1×10^{-2}
1.6×10^{-19}	509	227.77	25588	1.1×10^{-1}

transient being included in the statistics. Discarding the first twenty cycles removed it. A measurement floor that coincides with the signal is indistinguishable from a result, which is precisely why the floor was characterized before the sweep and not after.

The scaling law holds where it should. Thermal jitter accumulates as the square root of the noise density, so quadrupling the density must double σ . From 1×10^{-20} to 4×10^{-20} the measurement gives a factor of 2.01 against a predicted 2.00. That agreement is the evidence that what is being measured is the injected noise and not an artefact of the solver.

And it breaks at the top of the sweep, visibly. At 1.6×10^{-19} , sixteen times the reference density, σ grows by a factor of 21.3 where the law predicts 4.0, and the mean period drops from 230.4 to 227.8 ps. A zero-mean noise source cannot shift a mean period. The point is therefore outside the regime the model describes — the perturbation is large enough to change the operating point of the inverters — and it is reported rather than dropped, because the boundary of validity of a model is itself a result. Only the two central points are used for anything downstream.

7.3 Operating point: divider and throughput

The stochastic model fixes the target. For the PTG.2 thresholds of AIS 20/31 v3.0, the min-entropy criterion ($H_\infty \geq 0.98$) demands $Q \geq 0.2286$, and is more demanding than the Shannon criterion ($H \geq 0.9998$, $Q \geq 0.2022$); the design therefore targets the former, with $Q = 0.30$ as the working set-point, which yields $H_\infty = 0.9951$. Two properties of the model were established rather than assumed: the truncated series of equation 14 in [10] *overestimates* entropy and is thus not a lower bound, so the exact wrapped-Gaussian computation is used instead; and the first-harmonic derivation of min-entropy agrees with the exact one to six digits over the whole range of interest.

Since $Q_{\text{gen}} = K_D \cdot Q_{\text{raw}}$, the divider follows from the jitter that the campaign measures. Table 7.3 gives the operating points for the jitter range expected of this technology [8], assuming RO2 at 500 MHz.

Table 7.3: Divider and resulting throughput per ring for $Q \geq 0.2286$ [ESTIMATED].

σ/T per period	Q_{raw}	K_D	Throughput
7.1×10^{-4}	5×10^{-7}	457 200	1.09 kbit/s
1.4×10^{-3}	2×10^{-6}	114 300	4.37 kbit/s
2.8×10^{-3}	8×10^{-6}	28 575	17.5 kbit/s

These bracket the published design point of Fischer and Lubicz [13] ($K_D \approx 430\,000$)

for $\sigma = 5.01$ ps on $T = 7.81$ ns), which is the sanity check that the model is being applied as its authors intended.

The architectural consequence is stated plainly: a single ERO delivers kbit/s, not Mbit/s. That is the price of demonstrable entropy, and it is affordable here because the engine’s product is a 256-bit seed for the Deterministic Random Bit Generator (DRBG), not a bit stream. More throughput means more independent rings, each with its own bound, not a faster ring.

7.4 Functional verification, consolidated

Chapter 5 describes the strategy; this section records the totals. Every figure in Table 7.4 was obtained by running the suite, not by reading a previous log.

Table 7.4: Verification totals [SIMULATED].

Suite	What it checks
VHDL testbenches (GHDL)	rings, ERO, capture, health tests, I2C, Advanced Encryption Standard (AES), C
<code>ctr_drbg_ref.py</code>	CAVP vectors, with and without derivation function
<code>ero_model.py</code>	stochastic model, exactness and bounds
<code>jitter_estimator.py</code>	jitter recovery on synthetic data of known Q
<code>cortes_salud.py</code>	health-test cutoffs against SP 800-90B formulas
<code>verifica_top.py</code>	cross-check of the top-level bench with an independent AES

Two of these deserve a sentence. The 960 CAVP cases are the strongest single piece of evidence in the work: the generator is not merely self-consistent, it reproduces the outputs that National Institute of Standards and Technology (NIST) publishes for CTR_DRBG with AES-128, in both the derivation-function and the raw-entropy configurations. And the health-test cutoffs, $C_{RCT} = 22$ and $C_{APT} = 596$, had been computed by hand from the standard and were never checked; deriving them again from the formulas of SP 800-90B [6] confirms both. The same check quantifies the cost of the complement test added to the Adaptive Proportion Test (APT), which is an addition of this work and not of the standard: it raises the false-alarm rate per window from $2^{-20.15}$ to $2^{-20.14}$, a 0.8% increase, because the complementary tail lies 5.7 standard deviations away. The cutoff therefore remains valid as it stands and the addition is free in practice.

7.5 Comparison with commercial secure elements

The engine occupies the same functional slot as a commercial secure element: a small companion device that generates and holds keys for a host microcontroller over a two-wire bus. Table 7.5 places it beside the three parts that dominate the I2C secure-element market.

Table 7.5: The engine against commercial secure elements. Certification data from the manufacturers’ datasheets.

	ATECC608B [7]	SE050 [64]	OPTIGA Trust M [65]	This work
Bus	I2C	I2C	I2C	I2C
Certification	FIPS 140-3 L2	CC EAL6+	CC EAL6+	none
Entropy source	undisclosed	undisclosed	undisclosed	ERO, published
Entropy bound	not published	not published	not published	derived from measured jitter
Key export	not possible	not possible	not possible	test mode only
Inspectable	no	no	no	yes, RTL released

The honest reading of that table has two halves.

On capability and assurance the commercial parts win, and it is not close. Two of them carry CC EAL6+ evaluations, which represent an amount of third-party scrutiny — including physical attack resistance — that no academic project reproduces. They offer asymmetric cryptography, certified provisioning flows and manufacturing support. This work offers symmetric key generation and authentication, and carries no certification at all.

On what the thesis is actually about, the comparison inverts. In all three commercial devices the entropy source is undisclosed: the datasheets state that a True Random Number Generator (TRNG) is present, not what it is, and no min-entropy bound derived from a measured physical parameter is published for any of them. Their randomness is trusted because the vendor and the evaluation scheme are trusted. The engine described here makes the opposite trade: it has no certificate, and it can show where each bit of entropy comes from, what physical measurement bounds it, and what the bound becomes when the temperature or the divider changes. That is the same distinction that the author’s earlier characterization of the ESP32 generator ran into from the other side — a black box can be tested statistically but cannot be bounded — and it is the reason this engine was built as a white box.

Which of the two is preferable depends on the question being asked. For shipping a product, the certified part. For knowing what the randomness in that product is worth, only the open one answers.

7.6 Uncertainty budget

Four quantities carry the conclusions, and each has a different and declared kind of uncertainty.

Simulated jitter. Resolution floor 1.38 fs [SIMULATED], about 870 times below the smallest quantity measured. The scaling check against the expected square-root

law agrees to 0.5 % in the valid regime, and the regime boundary is documented in Section 7.2. The figure depends on the noise density assumed for the technology, which is itself [ESTIMATED]: what the simulation establishes is the *method* and the ring’s response, not the absolute jitter of a fabricated device.

Recovered Q . The estimator reproduces a known Q with a bias below 2 % and a dispersion of 4.0 % over eight seeds with 20 000 blocks [SIMULATED]. Without the corrections applied here a plain straight-line fit biases the result by -18% , so the correction is not cosmetic.

Area. A lower bound [SIMULATED]: cells only, flattened, without routing, fill or clock tree. Published growth factors from cell area to core area on this library are ≈ 2.35 for an AES, and the optimization savings of Chapter 6 are [ESTIMATED] and explicitly not claimed as measured.

Min-entropy of the built device. Not yet bounded. The model, the estimator and the cutoffs are ready; the number requires the campaign of Section 7.7. No value is offered here, and none should be inferred from the simulated jitter.

7.7 The pending campaign

The one result that silicon must supply is the min-entropy of the rings as built, and the procedure is fixed in advance so that it cannot be tuned after seeing the data.

1. **Confirm the rings oscillate.** Read the frequency of each ring with the embedded counter before trusting any entropy measurement. A ring removed by the synthesizer, or two rings that have locked to each other, would otherwise be indistinguishable from a working source.
2. **Measure the jitter** with $K_D = 1$ by the method of Fischer and Lubicz [13], sweeping the pair distance M and fitting the linear region, with at least three rings so that the global component separates from the local one [31]. Fit $\sigma^2(t) = at + bt^2$ and keep only the thermal term.
3. **Fix K_D** from the measured jitter for $Q \geq 0.2286$ with margin, and recompute the health-test cutoffs for the measured H_∞ rather than the assumed 0.98, using `cortes_salud.py`.
4. **Collect raw bits** at that operating point and run the NIST SP 800-90B estimators, contrasting the empirical estimate with the model’s bound. Agreement supports the model; disagreement is a result about the model’s hypotheses, and either way it is reported.

5. **Repeat across placement** — rings in separated regions versus adjacent ones — to quantify the coupling that the literature warns about, which is the measurement the architecture of this engine was chosen to make possible.

Until step 4 closes, the engine’s claim is precise and limited: the chain from physical noise to key material is designed, implemented and verified against the standards, and the bound it will produce is computable from a measurement that has not yet been taken.

Chapter 8

Conclusions and Future Work

8.1 Conclusions

8.2 Contributions

8.3 Future work

Bibliography

- [1] A. T. Markettos and S. W. Moore, “The frequency injection attack on ring-oscillator-based true random number generators,” in *Cryptographic Hardware and Embedded Systems – CHES 2009*, ser. LNCS, vol. 5747. Springer, 2009, pp. 317–331, [VERIFICADO] PDF + Crossref; keyspace $2^{64} \rightarrow 3300$; tarjeta EMV 2004; enganche 74HC04 a 24 MHz / 900 mV pk-pk. [Online]. Available: <https://www.iacr.org/archive/ches2009/57470316/57470316.pdf>
- [2] D. Hurley-Smith and J. Hernandez-Castro, “Certifiably biased: An in-depth analysis of a common criteria EAL4+ certified TRNG,” *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 4, pp. 1031–1041, 2018, [VERIFICADO-META] Crossref.
- [3] J. Blanco-Romero, F. Almenares Mendoza, D. Díaz-Sánchez, and A. Marín-López, “Entropy bootstrapping for wireless embedded systems,” arXiv:2607.08865, Sep. 2026, [VERIFICADO] página abs; ESP32 RNG pseudoaleatorio con RF apagada pasa cribados estadísticos. [Online]. Available: <https://arxiv.org/abs/2607.08865>
- [4] National Institute of Standards and Technology, “Decision to revise NIST SP 800-22 Rev. 1a,” Apr. 2022, [VERIFICADO] Cita literal: “rejecting its use for assessing cryptographic random number generators”. [Online]. Available: <https://www.nist.gov/news-events/news/2022/04/decision-revise-nist-sp-800-22-rev-1a>
- [5] M. Peter and W. Schindler, “A proposal for functionality classes for random number generators, version 3.0,” Bundesamt für Sicherheit in der Informationstechnik (BSI), Mathematical-technical reference of AIS 20/31, Sep. 2024, [VERIFICADO] PDF 306 pp.; PTG.2.2: Shannon ≥ 0.9998 y/o min-entropía ≥ 0.98 , Prob(1) en (0.493,0.507); §4.5.3 online test, §4.5.4 total failure test, §4.6 suite T_{irr} . [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Certification/Interpretations/AIS_31_Functionality_classes_for_random_number_generators_e_2024.pdf
- [6] M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, M. L. Baish, and M. Boyle, “Recommendation for the entropy sources used for random bit generation,” NIST,

- Special Publication SP 800-90B, Jan. 2018, [VERIFICADO: PDF extraído; sección 3.1.5 completa (lista de componentes vetted, Tabla 1, fórmula Output_Entropy, factor 0,999 para no vetted). Autores citados de memoria]. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/90/b/final>
- [7] *ATECC608B CryptoAuthentication Device Summary Data Sheet*, Microchip Technology Inc., 2021, [VERIFICADO] Cita literal: "The random number generator is designed to meet the requirements documented in the NIST 800-90A, 800-90B and 800-90C documents" — redacción de intención de diseño, no de certificación. AES-128 únicamente; solo curva NIST P-256; 16 slots de clave; 1.208 B de data zone; I2C hasta 1 Mbps + SWI; sin SPI. Es un datasheet RESUMEN: el completo está bajo NDA. [Online]. Available: <https://ww1.microchip.com/downloads/aemDocuments/documents/SCBU/ProductDocuments/DataSheets/ATECC608B-CryptoAuthentication-Device-Summary-Data-Sheet-DS40002239B.pdf>
- [8] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, "A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices," in *26th International Conference on Field Programmable Logic and Applications (FPL 2016)*, 2016, pp. 1–10, [VERIFICADO] PDF HAL completo + Crossref; Tabla II (Spartan-6, Cyclone V, SmartFusion2); jitter 2–4 ps; ec. (1) H_{min} . [Online]. Available: <https://hal.science/hal-01382955/document>
- [9] D. Lubicz and V. Fischer, "Recommendations for the design and validation of a physical true random number generator integrated in an electronic device, version 1.0," IACR Cryptology ePrint Archive, Report 2024/301, Feb. 2024, [VERIFICADO] PDF 33 pp.; "proven entropy"; ERO como ejemplo canónico; comparación con AIS 20/31 y SP 800-90B. [Online]. Available: <https://eprint.iacr.org/2024/301.pdf>
- [10] M. Baudet, D. Lubicz, J. Micolod, and A. Tassiaux, "On the security of oscillator-based random number generators," *Journal of Cryptology*, vol. 24, no. 2, pp. 398–425, 2011, [VERIFICADO] ePrint 2009/299 leído; Crossref confirma DOI/vol/págs. $Q = \sigma^2 \Delta t$; Prop. 1 ecs. 8, 10, 12; Cor. 1 ec. 14. [Online]. Available: <https://eprint.iacr.org/2009/299.pdf>
- [11] W. Killmann and W. Schindler, "A design for a physical RNG with robust entropy estimators," in *Cryptographic Hardware and Embedded Systems – CHES 2008*, ser. LNCS, vol. 5154. Springer, 2008, pp. 146–163, [VERIFICADO-META] Crossref.
- [12] Y. Ma, J. Lin, T. Chen, C. Xu, Z. Liu, and J. Jing, "Entropy evaluation for oscillator-based true random number generators," in *Cryptographic Hardware and Embed-*

- ded Systems – CHES 2014*, ser. LNCS, vol. 8731. Springer, 2014, pp. 544–561, [VERIFICADO-META] Crossref.
- [13] V. Fischer and D. Lubicz, “Embedded evaluation of randomness in oscillator based elementary TRNG,” in *Cryptographic Hardware and Embedded Systems – CHES 2014*, ser. LNCS, vol. 8731. Springer, 2014, pp. 527–543, [VERIFICADO] PDF completo; Algoritmo 1; error $< 5\%$; $\sigma = 5.01$ ps / 7.81 ns en Cyclone III; $K_D \approx 430\,000$ para $H \geq 0.997$. [Online]. Available: <https://www.iacr.org/archive/ches2014/87310109/87310109.pdf>
- [14] B. Sunar, W. J. Martin, and D. R. Stinson, “A provably secure true random number generator with built-in tolerance to active attacks,” *IEEE Transactions on Computers*, vol. 56, no. 1, pp. 109–119, 2007, [VERIFICADO-META] Crossref.
- [15] K. Wold and C. H. Tan, “Analysis and enhancement of random number generator in FPGA based on oscillator rings,” *International Journal of Reconfigurable Computing*, vol. 2009, p. Article ID 501672, 2009, [VERIFICADO-META] Crossref; 100 Mbit/s, < 100 LE Cyclone II (abstract).
- [16] N. Bochard, F. Bernard, V. Fischer, and B. Valtchanov, “True-randomness and pseudo-randomness in ring oscillator-based true random number generators,” *International Journal of Reconfigurable Computing*, vol. 2010, pp. Article ID 879281, 1–13, 2010, [VERIFICADO-META] Crossref.
- [17] P. Kohlbrenner and K. Gaj, “An embedded true random number generator for FPGAs,” in *Proc. ACM/SIGDA 12th International Symposium on Field Programmable Gate Arrays (FPGA 2004)*, 2004, pp. 71–78, [VERIFICADO] PDF + Crossref; Virtex XCV1000, 0.5 Mbit/s, diferencias de periodo 22–35 ps. [Online]. Available: https://people-ece.vse.gmu.edu/~kgaj/publications/conferences/GMU_FPGA_2004.pdf
- [18] B. Valtchanov, V. Fischer, and A. Aubert, “Enhanced TRNG based on the coherent sampling,” in *3rd International Conference on Signals, Circuits and Systems (SCS 2009)*, 2009, pp. 1–6, [VERIFICADO-META] Crossref.
- [19] A. Peetermans, V. Rožić, and I. Verbauwhede, “Design and analysis of configurable ring oscillators for true random number generation based on coherent sampling,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 14, no. 2, pp. 7:1–7:20, 2021, [VERIFICADO] PDF COSIC + Crossref; modelo COSO ecs. 1–10; Spartan-7 4.65 Mbit/s; umbral min-entropía 0.91 (interpretación de los autores). [Online]. Available: <https://cosicdatabase.esat.kuleuven.be/backend/publications/files/journal/3280>

- [20] —, “A highly-portable true random number generator based on coherent sampling,” in *29th International Conference on Field Programmable Logic and Applications (FPL 2019)*, 2019, pp. 218–224, [VERIFICADO] PDF Lirias + Crossref; Spartan-6 3.30 Mbit/s, SmartFusion2 1.47 Mbit/s, AIS-31 sin post-procesado. [Online]. Available: <https://lirias.kuleuven.be/retrieve/0ee069b3-afb3-43e3-9fe3-9e6d0b90ae46>
- [21] M. Varchola and M. Drutarovský, “New high entropy element for FPGA based true random number generators,” in *Cryptographic Hardware and Embedded Systems – CHES 2010*, ser. LNCS, vol. 6225. Springer, 2010, pp. 351–365, [VERIFICADO] PDF + Crossref; TERO en un CLB de Spartan-3E. [Online]. Available: <https://www.iacr.org/archive/ches2010/62250341/62250341.pdf>
- [22] F. Bernard, P. Haddad, V. Fischer, and J. Nicolai, “From physical to stochastic modeling of a TERO-based TRNG,” *Journal of Cryptology*, vol. 32, no. 2, pp. 435–458, 2019, [VERIFICADO-META] Crossref.
- [23] A. Cherkaoui, V. Fischer, L. Fesquet, and A. Aubert, “A very high speed true random number generator with entropy assessment,” in *Cryptographic Hardware and Embedded Systems – CHES 2013*, ser. LNCS, vol. 8086. Springer, 2013, pp. 179–196, [VERIFICADO] PDF + Crossref; STR; ecs. 8–9 cota H_m ; 200 Mbit/s; jitter 2 ps Cyclone III, 2.5 ps Virtex-5. [Online]. Available: <https://www.iacr.org/archive/ches2013/80860154/80860154.pdf>
- [24] B. Yang, V. Rožić, M. Grujić, N. Mentens, and I. Verbauwhede, “ES-TRNG: A high-throughput, low-area true random number generator based on edge sampling,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 3, pp. 267–292, 2018, [VERIFICADO] página TCHES; Spartan-6: 10 LUT + 5 FF, 1.15 Mbit/s, Shannon 0.997. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/7276>
- [25] V. Rožić, B. Yang, W. Dehaene, and I. Verbauwhede, “Highly efficient entropy extraction for true random number generators on FPGAs,” in *Proc. 52nd Annual Design Automation Conference (DAC 2015)*, 2015, pp. 116:1–116:6, [VERIFICADO-META] Crossref; DC-TRNG con cadena de acarreo.
- [26] CEA-Leti, “Opentrng: open-source reference ptrngs based on ring oscillators (ero, muro, coso) — hardware documentation and repository,” <https://opentrng.org/docs/hardware.html> ; <https://github.com/opentrng/ptrng>, 2025, [VERIFICADO] Licencia MIT; Digilent Arty A7 (xc7a35ticsg324-1L); LUT2 NAND + LUT1 inversores; constraints.xdc con área prohibida; sin cifras de entropía publicadas.

- [27] W. Schindler and W. Killmann, “Evaluation criteria for true (physical) random number generators used in cryptographic applications,” in *Cryptographic Hardware and Embedded Systems – CHES 2002*, ser. LNCS, vol. 2523. Springer, 2003, pp. 431–449, [VERIFICADO-META] Crossref.
- [28] P. Haddad, Y. Teglia, F. Bernard, and V. Fischer, “On the assumption of mutual independence of jitter realizations in P-TRNG stochastic models,” in *Design, Automation & Test in Europe (DATE 2014)*, 2014, pp. 1–6, [VERIFICADO-META] Crossref.
- [29] M.-J. O. Saarinen, “On entropy and bit patterns of ring oscillator jitter,” arXiv:2102.02196 [cs.CR], v4, Oct. 2021, [VERIFICADO] PDF; advierte que la cota de Baudet nunca baja de 0.415; método numérico exacto con gaussiana envuelta. [Online]. Available: <https://arxiv.org/abs/2102.02196>
- [30] V. Fischer, F. Bernard, N. Bochard, and M. Varchola, “Enhancing security of ring oscillator-based TRNG implemented in FPGA,” in *International Conference on Field Programmable Logic and Applications (FPL 2008)*, 2008, pp. 245–250, [VERIFICADO-META] Crossref; “random jitter accumulates slower than deterministic jitter” (abstract).
- [31] D. Lubicz and M. Skórski, “Improved phase noise measurement for multi-ring oscillator based TRNG (arxiv title: Quantifying jitter transfer for differential measurement: Enhancing security of oscillator-based TRNGs),” arXiv:2410.08259 [cs.CR], Oct. 2024, [VERIFICADO] PDF completo; jitter transfer principle; dos métodos con tres osciladores; Cyclone V. [Online]. Available: <https://arxiv.org/abs/2410.08259>
- [32] L. Benea, M. Carmona, V. Fischer, F. Pebay-Peyroula, and R. Wacquez, “Impact of the flicker noise on the ring oscillator-based TRNGs,” in *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHEs)*, 2024, [VERIFICADO: PDF leído por el investigador; DOI pendiente de anotar. Datos y scripts abiertos en github.com/opentrng/papers].
- [33] D. Lubicz and N. Bochard, “Towards an oscillator based TRNG with a certified entropy rate,” *IEEE Transactions on Computers*, vol. 64, no. 4, pp. 1191–1200, 2015, [VERIFICADO-META] Crossref; contenido según abstract.
- [34] W. Killmann and W. Schindler, “A proposal for: Functionality classes for random number generators, version 2.0,” Bundesamt für Sicherheit in der Informationstechnik (BSI), Tech. Rep., Sep. 2011, [VERIFICADO] PDF 133 pp.; PTG.2.7: “average Shannon entropy per internal random bit exceeds 0.997”. [Online].

- Available: https://cosec.bit.uni-bonn.de/fileadmin/user_upload/teaching/15ss/15ss-taoc/01_AIS31_Functionality_classes_for_random_number_generators.pdf
- [35] Bundesamt für Sicherheit in der Informationstechnik, “Transition policy to version 3.0 of the mathematical/technical reference, part of AIS 20/31, version 1.0,” BSI, Tech. Rep., Apr. 2025, [VERIFICADO] Certificaciones nuevas con AIS antigua hasta 31/03/2026. [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Certification/Interpretations/AIS_20_31_Transition_Policy_Version_3_0.pdf
- [36] W. Schindler, “The new AIS 20/31,” Presentación, European Cyber Week 2024, Rennes, Nov. 2024, [VERIFICADO] “At most 0.9998 bit Shannon entropy / 0.98 bit min-entropy can be claimed on the basis of the stochastic model”. [Online]. Available: <https://www.creachlabs.fr/sites/default/files/public/media/document/2024-12/ecw2024-rng-puf-schindlerwerner-20241120.pdf>
- [37] National Institute of Standards and Technology, “Sp800-90b_entropyassessment: C++ implementation of the min-entropy assessment methods of sp 800-90b,” GitHub repository, 2018, [VERIFICADO] README: C++11/OpenMP, libbz2/libdivsufsort/jsoncpp/OpenSSL/MPFR/GMP; ea_iid, ea_non_iid, ea_restart, ea_conditioning, ea_transpose; consultado 2026-09-11. [Online]. Available: https://github.com/usnistgov/SP800-90B_EntropyAssessment
- [38] L. E. Bassham, A. L. Rukhin, J. Soto, J. R. Nechvatal, M. E. Smid, E. B. Barker, S. D. Leigh, M. Levenson, M. Vangel, D. L. Banks, N. A. Heckert, J. F. Dray, and S. Vo, “A statistical test suite for random and pseudorandom number generators for cryptographic applications,” National Institute of Standards and Technology, NIST Special Publication 800-22 Rev. 1a, Apr. 2010, [VERIFICADO-META] Crossref.
- [39] D. Hurley-Smith and J. Hernandez-Castro, “Great expectations: A critique of current approaches to random number generation testing & certification,” in *Security Standardisation Research (SSR 2018)*, ser. LNCS, vol. 11322. Springer, 2018, pp. 143–163, [VERIFICADO-META] Crossref.
- [40] “ESP-IDF programming guide: Random number generation (ESP32),” Espressif Systems, 2026, [VERIFICADO] Condiciones RF / bootloader_random_enable / bootloader; “pseudo-random only” en caso contrario; esp_random() busy-wait. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/random.html>
- [41] *ESP32 Technical Reference Manual, Version 5.8, Chapter 18: Random Number Generator (RNG)*, Espressif Systems, 2025, [VERIFICADO] PDF; RNG_DATA_REG

- 0x3FF75144; ruido de SAR ADC / HS ADC via XOR; RC_FAST_CLK 8 MHz, 2 bits/ciclo; lectura máx. 500 kHz / 5 MHz; Dieharder 3.31.1 sobre 2 GB; pseudoaleatorio sin ADC. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [42] J. Castillo, P. Aran Vila, F. Palacio, B. Garrido, S. Hernández, and A. Cirera, “Automated framework for testing random number generators for IoT security applications using NIST SP 800-22,” *IoT*, vol. 7, no. 1, p. 26, 2026, [VERIFICADO-META] Crossref; ESP32-C3, 2 GB, 1000 secuencias de 10^6 bits, solo SP 800-22 (según resumen).
- [43] P. Hidayatulloh, N. S. Qolbiyah, A. Nugroho, H. Mukhtar, I. Purnama, and F. Simanjuntak, “Performance evaluation of ESP32 random number generator for LoRa communication security,” in *2024 International Conference on Radar, Antenna, Microwave, Electronics, and Telecommunications (ICRAMET)*, 2024, pp. 44–49, [VERIFICADO-META] Crossref; NIST STS (800-22).
- [44] National Institute of Standards and Technology, “Advanced encryption standard (AES),” NIST, Federal Information Processing Standards Publication FIPS 197-upd1, May 2023, publicado originalmente el 26-11-2001; actualización editorial (sin cambios técnicos) del 9-05-2023. [VERIFICADO: página CSRC y PDF, Tabla 3 y Apéndice D]. [Online]. Available: <https://csrc.nist.gov/pubs/fips/197/final>
- [45] T. Good and M. Benaissa, “AES on FPGA from the fastest to the smallest,” in *Cryptographic Hardware and Embedded Systems — CHES 2005*, ser. Lecture Notes in Computer Science, vol. 3659. Springer, 2005, pp. 427–440, 25 Gbit/s en Spartan-III XC3S2000 (pipelined); 124 slices + 2 BRAM y 2,2 Mbit/s en Spartan-II XC2S15 (ASIP 8 bit). [VERIFICADO: abstract leído del PDF].
- [46] P. Chodowiec and K. Gaj, “Very compact FPGA implementation of the AES algorithm,” in *Cryptographic Hardware and Embedded Systems — CHES 2003*, ser. Lecture Notes in Computer Science, vol. 2779. Springer, 2003, pp. 319–333, cifrado, descifrado y key schedule en 222 slices y 3 Block RAM de un Spartan-II XC2S30. [VERIFICADO: título/autores/DOI/páginas por Crossref; cifras por el resumen de Springer].
- [47] A. Moradi, A. Poschmann, S. Ling, C. Paar, and H. Wang, “Pushing the limits: A very compact and a threshold implementation of AES,” in *Advances in Cryptology — EUROCRYPT 2011*, ser. Lecture Notes in Computer Science, vol. 6632. Springer, 2011, metadatos [VERIFICADO] vía API de Semantic Scholar; cifra de 2.400 GE [VERIFICADO] en el resumen público. AES-128 SOLO CIFRADO en librería UMC 180 nm — que es justamente lo que necesita

- un CTR_DRBG. [NO VERIFICADO] rango de páginas. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-20465-4_6
- [48] A. Satoh, S. Morioka, K. Takano, and S. Munetoh, “A compact Rijndael hardware architecture with S-Box optimization,” in *Advances in Cryptology — ASIACRYPT 2001*, ser. Lecture Notes in Computer Science, vol. 2248. Springer, 2001, pp. 239–254, s-box en campo compuesto; 5,4 kpuertas en CMOS 0,11 μ m. [VERIFICADO: título/autores/DOI vía Springer Link; cifra de puertas por resumen, NO VERIFICADA en el texto].
- [49] D. Canright, “A very compact S-box for AES,” in *Cryptographic Hardware and Embedded Systems — CHES 2005*, ser. Lecture Notes in Computer Science, vol. 3659. Springer, 2005, pp. 441–455, 432 elecciones de base (polinomiales y normales) para los subcampos; mejora del 20 % sobre Satoh et al. [VERIFICADO: abstract leído del PDF de IACR]. [Online]. Available: <https://www.iacr.org/archive/ches2005/032.pdf>
- [50] J. Boyar and R. Peralta, “A new combinational logic minimization technique with applications to cryptology,” in *Experimental Algorithms (SEA 2010)*, ser. Lecture Notes in Computer Science, vol. 6049. Springer, 2010, pp. 178–189, circuito de la S-box de AES con el menor número de puertas conocido entonces. [VERIFICADO: cita y abstract en la página de NIST; el desglose 32 AND + 83 XOR NO VERIFICADO]. [Online]. Available: <https://www.nist.gov/publications/new-combinational-logic-minimization-technique-applications-cryptology>
- [51] H. Hadipour, “AES-VHDL: VHDL implementation of AES algorithm,” <https://github.com/hadipourh/AES-VHDL>, 2026, vHDL, GPL-3.0; 1 ronda/ciclo (10 ciclos); Artix-7 xc7a200t-3: cifrador 1104 LUT, 264 FF, 294,4 MHz, 3,77 Gbit/s; descifrador 1490 LUT, 253,8 MHz, 3,25 Gbit/s; verificado con vectores NIST. [VERIFICADO: README].
- [52] J. Strömbergson, “secworks/aes: Verilog implementation of the symmetric block cipher AES as specified in NIST FIPS 197,” <https://github.com/secworks/aes>, 2026, verilog 2001, BSD-2-Clause; iterativo a nivel de palabra, 4 S-box, 46 ciclos/bloque; Artix-7 200T-3: 2298 slices, 2989 FF, 97 MHz; Spartan-6: 2576 slices, 100 MHz. Testbench con vectores de FIPS 197 Ap. C y SP 800-38A. [VERIFICADO: README y tb_aes.v].
- [53] National Institute of Standards and Technology, “Secure hash standard (SHS),” NIST, Federal Information Processing Standards Publication FIPS 180-4, Aug. 2015, [VERIFICADO: página CSRC y PDF, secciones 5.1.1 y 6.2.2]. [Online]. Available: <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>

- [54] J. Strömbergson, “secworks/sha256: Hardware implementation of the SHA-256 cryptographic hash function,” <https://github.com/secworks/sha256>, 2026, verilog 2001, BSD-2-Clause; iterativo, 66 ciclos/bloque, sin relleno; Artix-7: 2471 LUT, 747 slices, 1930 FF, 108 MHz. Testbench con ejemplos oficiales de NIST. [VERIFICADO: README y tb_sha256.v].
- [55] E. Barker and J. Kelsey, “Recommendation for random number generation using deterministic random bit generators,” NIST, Special Publication SP 800-90A Rev. 1, Jun. 2015, [VERIFICADO: PDF extraído; Tablas 2 y 3, secciones 8.6, 10.1, 10.2.1.3.1 y 11 leídas. Autores citados de memoria]. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/90/a/r1/final>
- [56] J. Woodage and D. Shumow, “An analysis of NIST SP 800-90A,” in *Advances in Cryptology — EUROCRYPT 2019*, ser. Lecture Notes in Computer Science, vol. 11476. Springer, 2019, robustez de Hash_DRBG y HMAC_DRBG en el ROM (con salvedad); HMAC_DRBG sin additional_input no alcanza forward security; análisis práctico de fuga parcial de estado en CTR_DRBG. [VERIFICADO: abstract leído del PDF de IACR; DOI por Crossref]. [Online]. Available: <https://eprint.iacr.org/2018/349>
- [57] E. Barker, J. Kelsey, K. McKay, A. Roginsky, and M. S. Turan, “Recommendation for random bit generator (RBG) constructions,” NIST, Special Publication SP 800-90C, Sep. 2025, versión final (RBG1, RBG2, RBG3, RBGC). [VERIFICADO: página CSRC, estado y fecha; autores citados de memoria]. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/90/c/final>
- [58] M. Dworkin, “Recommendation for block cipher modes of operation: The CMAC mode for authentication,” NIST, Special Publication SP 800-38B (incl. updates as of 10-06-2016), Oct. 2016, publicado en mayo de 2005; erratas sustantivas del 6-10-2016 (frontal, retirada de ejemplos del Ap. D, bibliografía). [VERIFICADO: PDF extraído; autor según portada del PDF]. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/38/b/upd1/final>
- [59] J. Song, R. Poovendran, J. Lee, and T. Iwata, “The AES-CMAC algorithm,” IETF, RFC 4493, Jun. 2006, [VERIFICADO: vectores de prueba de la sección 4 confirmados; autores y fecha citados de memoria, NO VERIFICADOS en esta sesión]. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4493>
- [60] National Institute of Standards and Technology, “CAVP testing: Block ciphers — AES test vectors (KAT_AES.zip, aesmmt.zip, aesmct.zip),” <https://csrc.nist.gov/Projects/cryptographic-algorithm-validation-program/block-ciphers>, 2026, ficheros

- .rsp: ECBGFSbox128, ECBKeySbox128, ECBVarKey128, ECBVarTxt128, ECB-MMT128, CBCMMT128, etc. [VERIFICADO: página y zips descargados y listados].
- [61] —, “CAVP testing: Block cipher modes — CMAC test vectors (cmactestvectors.zip),” <https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/cavp-testing-block-cipher-modes>, 2026, contiene CMACGenAES128.rsp y CMACVerAES128.rsp. [VERIFICADO: página y zip descargado].
- [62] —, “CAVP testing: Random number generators — DRBG test vectors (drbgtestvectors.zip),” <https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/random-number-generators>, 2026, tres zips (no_reseed, pr_false, pr_true) con CTR_DRBG.rsp, Hash_DRBG.rsp, HMAC_DRBG.rsp. [VERIFICADO: página y zip descargado].
- [63] —, “ACVP-Server: NIST automated cryptographic validation protocol server and sample JSON test vectors,” <https://github.com/usnistgov/ACVP-Server>, 2026, carpeta gen-val/json-files con ACVP-AES-ECB-1.0, ACVP-AES-CBC-1.0, CMAC-AES-1.0, SHA2-256-1.0, ctrDRBG-1.0, hashDRBG-1.0, hmacDRBG-1.0; ficheros prompt.json / expectedResults.json / internalProjection.json. Licencia NIST. [VERIFICADO: README y API de GitHub].
- [64] *SE050 Plug & Trust Secure Element — Product Data Sheet*, NXP Semiconductors, 2023, [VERIFICADO] "TRNG compliant to NIST SP800-90B" y "DRBG compliant to NIST SP800-90A"; PRNG clase DRG.4 (AIS20) inicializado por TRNG clase PTG.2 (AIS31). AES-128/192/256; SHA-1 a SHA-512; ECC P-192 a P-521, Ed25519, X25519; RSA hasta 4096; 50 kB de flash segura; I2C hasta 3,4 MHz, dirección 0x48; sin SPI. CC EAL6+ "up to OS level". [Online]. Available: <https://www.nxp.com/docs/en/data-sheet/SE050-DATASHEET.pdf>
- [65] *OPTIGA Trust M Datasheet — SLS 32AIA010MH/S/K/L/M*, Infineon Technologies AG, 2024, [VERIFICADO] CERO menciones de NIST SP 800-90, AIS31 o AIS20 en todo el datasheet: solo declara "TRNG, DRNG" sin caracterizar. AES-128/192/256; SHA-256; ECC P-256/384/521 y Brainpool; RSA 1024/2048; 10 kB de memoria de usuario (4 ECC + 2 RSA + 1 AES + 4 X.509); I2C hasta 1 MHz, dirección 0x30; sin SPI. CC EAL6+ solo del HARDWARE (BSI-DSZ-CC-0961); PSA Level 3. [Online]. Available: <https://www.infineon.com/assets/row/public/documents/30/49/infineon-optiga-trust-m-sls32aia-datasheet-en.pdf>
- [66] *STSAFE-A110: Authentication, State-of-the-Art Security for Peripherals and IoT Devices — Datasheet*, STMicroelectronics, 2019, [VERIFICADO vía espejo oficial de Farnell, <https://www.farnell.com/datasheets/2917358.pdf>; st.com dio timeout

- reiterado] CERO menciones de NIST SP 800-90 o AIS31: solo un comando "Generate Random" sin caracterizar el generador. SHA-256/384; ECC P-256/P-384 NIST o Brainpool; sin RSA; 6 kB de EEPROM; I2C hasta 400 kbps, sin clock stretching; sin SPI. "CC EAL5+ AVA_VAN5 Common Criteria certified". [NO VERIFICADO] número de certificado CC y Protection Profile. [Online]. Available: <https://www.st.com/resource/en/datasheet/stsafe-a110.pdf>
- [67] DigiKey Electronics, "Precios y disponibilidad de ATECC608B, SE050, OPTIGA Trust M y STSAFE-A110," <https://www.digikey.com/>, 2026, [VERIFICADO] Precios en USD. ATECC608B-SSHDA-T (SOIC-8): 0,90 USD a cantidad 1; 0,8872 USD a 25; 0,725 USD a 4.000 en carrete. ATECC608B-MAHDA-T (UDFN-8): 0,87 USD a 1. SE050C2HQ1/Z01SDZ: 5,10 USD a 1. STSAFA110S8SPL02: 2,49 USD a 1. ADVERTENCIA: el tramo de 1.000 unidades a menudo NO existe; DigiKey salta de 250/500 al carrete completo. Ningún OPTIGA Trust M tiene precio a cantidad 1 en DigiKey (descatalogados o solo carrete).
- [68] National Institute of Standards and Technology, "Entropy source validation certificate #e46: ECC608 NRBG entropy source," <https://csrc.nist.gov/projects/cryptographic-module-validation-program/entropy-validations/certificate/46>, 2023, [VERIFICADO] Fuente física de osciladores de anillo, SP 800-90B, entropía por muestra 0,5071, laboratorio AEGISOLVE, validado 2023-09-28. Cubre ATECC608A/B/C. Es el ÚNICO certificado NIST de fuente de entropía física entre los cuatro elementos seguros comparados. No cubre SP 800-90A ni 800-90C.
- [69] I. Kuon and J. Rose, "Measuring the gap between FPGAs and ASICs," in *Proc. ACM/SIGDA 14th Int. Symp. on Field Programmable Gate Arrays (FPGA '06)*, 2006, pp. 21–30, [VERIFICADO por el investigador: PDF completo; 90 nm, area 40x, retardo 3.2x, potencia 12x].
- [70] K. Yang, D. Blaauw, and D. Sylvester, "An all-digital edge racing true random number generator robust against PVT variations," *IEEE Journal of Solid-State Circuits*, vol. 51, no. 4, pp. 1022–1031, 2016, [VERIFICADO por el investigador: DOI y cifras de robustez PVT].
- [71] P. Bayon, L. Bossuet, A. Aubert, V. Fischer, F. Poucheret, B. Robisson, and P. Maurine, "Contactless electromagnetic active attack on ring oscillator based true random number generator," in *Constructive Side-Channel Analysis and Secure Design (COSADE 2012)*, ser. LNCS, vol. 7275. Springer, 2012, pp. 151–166, [VERIFICADO-META] Crossref; 50 ROs enganchados por campo EM (abstract).
- [72] EUROPRACTICE, "Active EUROPRACTICE members," https://www.europractice.stfc.ac.uk/membership/membership_list.cfml, 2026, [VERIFICADO]

Figuran literalmente "Instituto de Microelectrónica de Sevilla (IMSE-CNM) CSIC" con membresía Full-IC y "Universidad de Sevilla" con Full-IC y FPGA-Only. Dato decisivo: no hace falta negociar acuerdo marco alguno.

- [73] EUROPRACTICE IC Service, “MPW schedules and prices 2026,” <https://europractice-ic.com/schedules-prices-2026/>, 2026, [VERIFICADO] Precios públicos para UMC, IHP, X-FAB, GlobalFoundries, ams OSRAM. EXCEPCIÓN: los MPW generales de TSMC exigen formulario "Request TSMC Prices" y STMicroelectronics figura "on request": esos precios NO son públicos.
- [74] —, “MPW fabrication and the mini@sic programme,” <https://europractice-ic.com/services/fabrication/>, 2026, [VERIFICADO] mini@sic descrito como compra de un bloque MPW estándar de fundición que se revende en sub-bloques menores y más baratos. Disponible para GlobalFoundries, IHP, TSMC, UMC y X-FAB.
- [75] IHP – Leibniz-Institut für innovative Mikroelektronik, “Open source PDK for SG13G2,” <https://www.ihp-microelectronics.com/services/research-and-prototyping-service/fast-design-enablement/open-source-pdk>, 2026, [VERIFICADO] SG13G2 es CMOS de 0,13 μm con HBT SiGe:C (fT hasta 350 GHz), doble óxido 1,2 V / 3,3 V, 5 metales finos + 2 gruesos, condensadores MIM. Incluye conjunto limitado de celdas lógicas estándar con vistas CDL, GDSII, LEF y Verilog. Flujo OpenROAD soportado. El PDK se califica de "early access version". [NO VERIFICADO] condiciones concretas del área MPW gratuita: no están publicadas.
- [76] Instituto de Microelectrónica de Sevilla (IMSE-CNM, CSIC–Universidad de Sevilla), “Áreas de investigación: Hardware security,” <http://www.imse-cnm.csic.es/es/areas/hardware-sec.php>, 2026, [VERIFICADO] Área formal que agrupa las líneas Cyber-security y Security and Reliability in CMOS and Emerging Technologies.
- [77] M. C. Martínez-Rodríguez, L. F. Rojas-Muñoz, E. Camacho-Ruiz, S. Sánchez-Solano, and P. Brox, “Efficient RO-PUF for generation of identifiers and keys in resource-constrained embedded systems,” *Cryptography*, vol. 6, no. 4, p. 51, 2022, [VERIFICADO] metadatos desde la ficha oficial de publicaciones del IMSE. [Online]. Available: <https://doi.org/10.3390/cryptography6040051>
- [78] P. Ortega-Castro, L. F. Rojas-Muñoz, J. M. Mora-Gutiérrez, P. Brox, and M. C. Martínez-Rodríguez, “VLSI integration of a RO-based PUF into a 65 nm technology,” in *IEEE Nordic Circuits and Systems Conference (NorCAS)*, 2024, [VERIFICADO] resumen vía API de Semantic Scholar. RO-PUF con metodología semi-custom en TSMC 65 nm, fabricado y caracterizado experimentalmente; robustez frente a temperatura y tensión. El resumen afirma que

- la estructura es adecuada tanto para FPGA como para ASIC. [Online]. Available: <https://doi.org/10.1109/NorCAS64408.2024.10752474>
- [79] P. Ortega-Castro, E. Camacho-Ruiz, J. M. Mora-Gutiérrez, P. Brox, and M. C. Martínez-Rodríguez, “Robust and scalable cell-based 65-nm CMOS RO-PUF implementation,” *IEEE Open Journal of the Solid-State Circuits Society*, 2026, early Access. [VERIFICADO] metadatos vía Crossref; [NO VERIFICADO] contenido técnico (IEEE Xplore devolvió HTTP 403). Referencia clave: enfoque "cell-based" con celdas estándar, exactamente el problema de rediseñar el RO al pasar de LUT a ASIC. [Online]. Available: <https://doi.org/10.1109/OJSSCS.2026.3689804>
- [80] J. M. Mora-Gutiérrez, C. J. Jiménez-Fernández, and M. Valencia-Barrero, “ASIC design and power characterization of standard and low power multi-radix Trivium,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 11, pp. 2682–2686, 2020, [VERIFICADO] metadatos desde la ficha oficial de publicaciones del IMSE. Prueba de que el instituto fabrica ASICs criptográficos, no solo prototipos en FPGA. [Online]. Available: <https://doi.org/10.1109/TCSII.2020.2969242>
- [81] IHP GmbH, “IHP-Open-PDK: Open source PDK for the IHP SG13G2 130 nm BiCMOS process,” <https://github.com/IHP-GmbH/IHP-Open-PDK>, 2026, [NO VERIFICADO] repositorio localizado en resultados de búsqueda; no se abrió el árbol de ficheros.
- [82] SkyWater Technology and Google, “SkyWater SKY130 PDK documentation: Foundry provided standard cell libraries,” <https://skywater-pdk.readthedocs.io/en/main/contents/libraries/foundry-provided.html>, 2026, [VERIFICADO] sky130_fd_sc_hd: site 0,46 x 2,72 um, 9 pistas met1, densidad bruta 266 kGates/mm² y "Routed Gate Density is 160 kGates/mm² or better" (factor 0,60). sky130_fd_sc_hdll: 200 kGates/mm². DATO CLAVE para convertir kGE en mm² y para cuantificar cuánto subestima la síntesis lógica.
- [83] GlobalFoundries, Google, and FOSSi Foundation, “GF180MCU open PDK: standard cell libraries for the GlobalFoundries 180 nm MCU process,” <https://github.com/fossi-foundation/gf180mcu-pdk>, 2026, [VERIFICADO] Licencia Apache 2.0. Librerías de celdas estándar de 7 y 9 pistas (gf180mcu_fd_sc_mcu7t5v0, gf180mcu_fd_sc_mcu9t5v0) más un juego de Oklahoma State University. Es la opción abierta de 180 nm, el nodo recomendado en el apartado 2.8.
- [84] Silicon Integration Initiative (Si2) and Silvaco (Nangate), “Open cell and free PDK libraries: Nangate 45 nm open cell library,” <https://si2.org/open-cell-and-free-pdk-libraries/>, 2026, [VERIFICADO] Librería genérica de celdas estándar bajo Apache 2.0 a través de Si2, gratuita para universidades y miembros.

ADVERTENCIA: no corresponde a ninguna fundición accesible, por lo que su área no es trasladable a un presupuesto de fabricación.

- [85] J.-P. Kaps, W. Diehl, M. Tempelmeier, E. Homsirikamol, K. Gaj *et al.*, “Hardware implementations of NIST lightweight cryptographic candidates: A first look,” in *NIST Lightweight Cryptography Workshop 2019*, 2019, métrica TPA en Mbps/LUT sobre Artix-7 xc7a100tcs324-3, Vivado 2018.3. [VERIFICADO]. [Online]. Available: <https://csrc.nist.gov/CSRC/media/Events/lightweight-cryptography-workshop-2019/documents/papers/hardware-implementations-of-nist-lwc-candidates-lwc2019.pdf>
- [86] Digilent Inc., *Basys3 FPGA Board Reference Manual (Revised August 12, 2014; applies to Basys3 rev. C)*, Digilent, 2014, xC7A35T-1CPG236C; 33 280 celdas lógicas en 5200 slices (4 LUT6 + 8 FF por slice); oscilador de 100 MHz en W5; Pmod JA/JB/JC + JXADC, 3,3 V en pines 6 y 12. Versión web actual: <https://digilent.com/reference/programmable-logic/basys-3/reference-manual> (no accesible en esta sesión). [VERIFICADO: PDF extraído]. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/university/aup-boards/XUPBasys3/documentation/Basys3_rm_8_22_2014.pdf
- [87] National Institute of Standards and Technology, “Block cipher modes of operation — CMAC mode for authentication: Example values (CMAC-AES128/192/256),” https://csrc.nist.gov/CSRC/media/Projects/Cryptographic-Standards-and-Guidelines/documents/examples/AES_CMAC.pdf, 2016, ejemplos retirados del Apéndice D de SP 800-38B en 2016. [VERIFICADO: PDF extraído; K1/K2 y etiquetas coinciden con RFC 4493].

Appendix A

Register Map

Appendix B

Official Test Vectors Used

Appendix C

Reproduction Guide